

Multi-robot path planning using improved particle swarm optimization algorithm through novel evolutionary operators

P.K. Das^{a,*}, P.K. Jena^b

^a Dept. Information Technology, VSSUT, Burla, Odisha, India

^b Dept. Mechanical Engineering, VSSUT, Burla, Odisha, India

ARTICLE INFO

Article history:

Received 8 July 2019

Received in revised form 14 March 2020

Accepted 13 April 2020

Available online 27 April 2020

Keywords:

IPSO-EOPs

Energy utilization

Path planning

Multiple mobile robots

Run time

Average untraveled trajectory target distance

ABSTRACT

The highlight of this paper is to propose an innovative approach to compute an optimal collision free trajectory path for each robot in a known and complex environment. The problem under consideration has been solved by employing an improved version of particle swarm optimization (IPSO) with evolutionary operators (EOPs). In the present context, PSO is improved with the concept of governance in human society and two evolutionary operators such as multi-crossover inherited from the genetic algorithm, and bee colony operator to enhance the intensification capability of the IPSO algorithm. The algorithm proposed to compute the deadlock free subsequent coordinate of an individual robot from their present coordinate, in addition, to minimize the path length for each robot by maintaining a good balance between intensification and diversification. Results obtained from the proposed IPSO-EOPs have been compared with competitors such as DE and IPSO in a similar environment to substantiate the robustness and usefulness of the algorithm. It perceives from the result obtained from simulation and experimentation that IPSO-EOPs is succeeding IPSO, and DE in terms of arrival time, generating a safe optimal path, and energy utilization during the travel.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

A robot is a controlled manipulator, capable of taking a decision, performing complex tasks like human beings [1]. A robot can be extremely precious in various precarious situations where humans are not capable to reach certain targets because of terrain locations and dangerous environments. The multi-robot System (MRS) employs a group of the robot to perform a certain task through collective behavior. A various complex situation that is incomprehensible for a single robot would be achieved through the combined behavior of MRS. There are a few anticipated advantages of MRS contrasted with single robot frameworks such as to increase the ability for resolving the complexity of the task, enhancing the performance, including fast task completion time, increase in robustness, reliability, and simplicity in the scheme. These advantages have attracted many researchers to investigate and create strong adaptable MRS by solving numerous challenging problems such as object tracking, cooperative formation, exploration, industrial transportation, cooperative task handling, target tracking, human material handling, a military mission to name just a few. The main objective of autonomous mobile robotics is to develop a physical framework that can

provide autonomy to robots for autonomous navigation tenaciously without the intervention of human operators in a cluttered known environment. Robot navigation is a process to find the goal location by keeping away from the obstacles in its pathway from the source position [2] by satisfying constraints like distance, energy and time [3–7]. This process consists of four separate modules: (1) perception, the robot extracts the necessary information through sensors; (2) localization, the robot determines to steer its position in the environment; (3) planning of the path, the robot decides its steering direction to achieve the goal position by avoiding obstacles; (4) motion control, adjust the motion to generate the required trajectory of the path [8–10]. The Robot Path planning has undergone a sequence of rotation and translation to reach the destination from their predefined initial position by keeping away from the obstacles on its pathway. The development of different meta-heuristic techniques to provide autonomy to mobile robots in path planning is one of the most challenging fields in the current exploration. This inclination is encouraged to overcome the current gap between the existing technologies and demands of the new application, e.g., the present industrial robots have low flexibility in autonomy. The main objective of the multiple mobile robots' path planning is to compute an optimal collision free trajectory path from predefined source to target for each robot without colliding with any teammates or obstacles in the workspace [11]. Depending on the position of the robot in the environment and techniques,

* Corresponding author.

E-mail addresses: daspradipta78@gmail.com (P.K. Das), phenavssut@gmail.com (P.K. Jena).

employed to resolve the path planning problem is presented in Fig. 1.

One of the most challenging problems in MRS is to compute an optimal collision free trajectory path for each robot to improve the system performance subjected to a set of constraints. One of the classical ways is to send the robot to discover its environment for localizing the landmark that can be used in path planning. In this situation, the robot trusts deeply on its sensors, map construction, and update its position. However, the environment consists of a large amount of uncertainty and mapping techniques take a large amount of memory, time and incapable to generate the optimality on the path.

In the last decades, several classical approaches have been developed for solving on MRS by the research community such as classical approaches are Road map method, potential field method, cell decomposition, Simulated Annealing [12], Voronoi diagram [13], visibility graph-based, A* Algorithm [14–16], neural network and so on. Most of the classical approaches have been given attention to optimizing the path distance. But, the following weaknesses are associated with the traditional approaches [17] such as (1) larger problem space, involves more computational complexity and (2) there is a great chance that the solution of the problem space under consideration may fall at local minima. Therefore, these weaknesses cause the traditional approaches wasteful in the various problem spaces. A direction of current research in the robot system focuses on not only an accurate control of robot manipulators but also empowering them with intelligence.

Hence, an intelligent technique such as computational intelligence begins with the underlying principle of artificial intelligence. Artificial Intelligence emulates human intelligence on machines so that the machines think like human beings and act like them. Thus, artificial intelligence helps to raise knowledge about reasoning, machine learning, planning, intelligent search and also is essential to cope with all types of uncertainty present in the environment.

In recent years, many researchers are paying attention to the evolutionary based problem solving techniques. Evolutionary robotics based path planning are generalized for any dynamic environment situations because many natural evolution techniques are mimic into the programming like neural network [18], Tree structure Encoding [19], fuzzy logic [20,21], gravitational search algorithm [22–24], particle swarm optimization [25,26], differential evolution [27], reinforcement learning [28], Artificial immune system [29], Artificial bee colony [30], hybridization of evolutionary computing such as IPSO-GSA [31], IPSO-DV [17], multi-objective variable neighborhood search [32] and other evolutionary algorithms [30,33].

Multi-objective Particle Swarm Optimization (MOPSO) is a variation of the PSO to solve multi-objective optimization problems [34]. Most of the multi-objective particle swarm optimization algorithms largely depend on the global or personal best particles stored in an external archive, the author has been used a competitive mechanism based multi-objective particle swarm optimizer, where the particles are updated based on the pairwise competitions performed in the current swarm at each generation. The performance of the proposed competitive multi-objective particle swarm optimizer is verified by benchmark comparisons with several state-of-the-art multi-objective optimizers. Infrared image segment and reconstruction of infrared image sequences have been carried out through fractional Darwin particle swarm optimization algorithm (FODPSO) [35]. Multi-level image thresholding method in which a Chaotic Darwinian Particle Swarm Optimization algorithm is applied to images compressed by using Fuzzy Transforms [36]. A dynamic group learning distributed particle swarm optimization

(DGLDPSO) has been exercised for the large-scale cloud workflow scheduling [37]. In this algorithm, first, the entire population is divided into many groups, and these groups are coevolved by using the master-slave multi-group distributed model, forming a distributed PSO (DPSO) to enhance the algorithm diversity. Second, a dynamic group learning(DGL) has been adopted for DPSO to balance the convergence and diversity. These techniques are recently increasingly used by researchers and scientist community for various application domains of handling uncertainty and imprecision with a reasonable amount of computational complexity.

Hybridization of IPSO-EOPs has been projected to solve the multi-robot path planning problem, in the multi-robot problem, each robot computes its next position through the evaluation of objective function using IPSO to compute an optimal collision free path towards their corresponding goals. The velocity of IPSO is adjusted using the Evolutionary operators. The objective function is designed using two parts for IPSO. The first one is the generation of the subsequent point for each robot and the next one consists of a set of constraints that ensure colleagues, non-static and static obstacles do not collide with the robot.

The main emphasized of the work may be précised as : (i) The problem has expressed as multi-objective optimization with different constraints and resolve the problem through hybridization of IPSO-EOPs; (ii) the problem in the paper aims is to generate a collision free optimal trajectory path for each robot through the IPSO-EOPs by optimizing the energy utilization in the form of number turns essential to attain at the destination; (iii) the effectiveness and robustness of the our proposed algorithm has been validated through real khepera-II robot and simulation; (iv) the results obtained through the proposed algorithm is effectively validated by comparing the result obtained from its opponents such as IPSO and DE in simulation and real platform.

The main contribution of the paper highlights the implementation of the IPSO-EOPs to plane the path for multiple mobile robots in a complex environment. In this problem, the start position and the goal position are known to every robot. The task of each robot is to generate a collision free optimal trajectory path from a known source to their target by satisfying the environmental constraints and optimizing the different parameters which have been discussed in detail in the subsequent section. During the path generation, each robot computes its next local optimal coordinate in a stepwise manner. Here, the problem is considered as an optimization problem. The objective (fitness) function for the above problem has been constructed by considering three different criteria such as the Euclidean distance between the current position and a target position, avoiding of static obstacles, prediction of movable (dynamic) obstacles and path smoothness. The multi-objective function is formulated as a single objective function by the weighted sum of each objective function. The hybrid techniques have freshly established significant attention in the field of path planning in the multi-robot environment. In this work, we have used the projected IPSO-EOPs technique by modifying the basic PSO and hybridized with evolutionary operators to generate an optimal collision free path for each robot.

The flexibility and applicability of the proposed algorithm are validated through the real platform and simulation. The result demonstrates that the offered technique can enhance the quality of the solution in a rational quantity of time and the hybrid IPSO-EOPs technique is designed to generate an optimal path successfully by avoiding both static and dynamic obstacles on its path towards the goal. Finally, simulation and real platform results are demonstrated and compared with previously published results. The problem under current study can be further improved as follows (1) the robustness of the current method was assessed

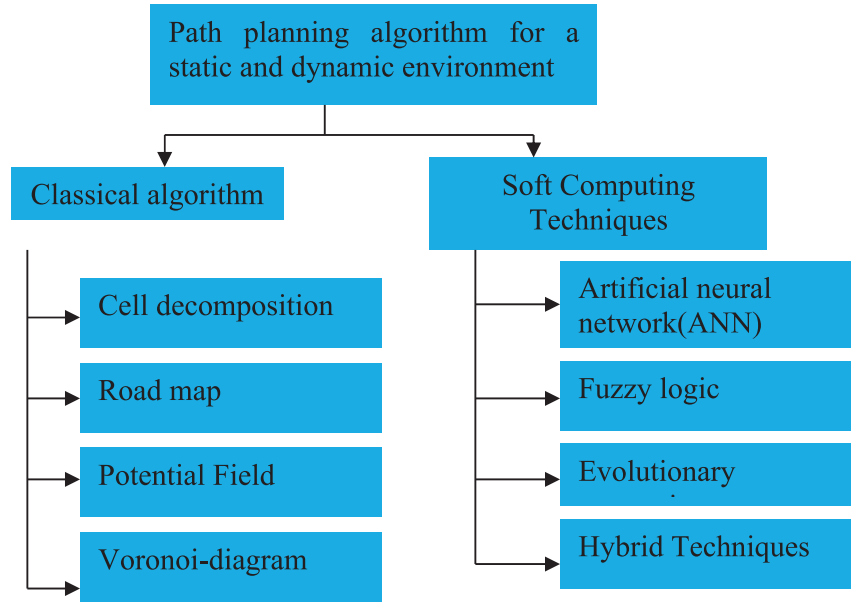


Fig. 1. Classification of path planning based on the type of environment.

by considering important sets of parameters, (2) the selected parameters were restricted by the search method and it could be improved significantly using more complex search schemes.

The rest part of this work is structured as follows. The Basic Particle swarm optimization and its Improvement are presented briefly in Section 2. The proposed Evolutionary operators are presented in Section 3. Hybrid IPSO-EOPs for multiple mobile robot path planning is outlined in Section 4. The mathematical design of multiple mobile robots is explained in Section 5. Section 6 presents a detail description of the path planning algorithm. The outcomes of the simulation framework are outlined in Section 7. In Section 8, the experiment conducted through the real platform is presented. The comparative analysis of the proposed work with other related works has been explained in Section 9 and finally, Section 9 ends with conclusions and future direction of the proposed work.

2. Particle swarm optimization (PSO)

2.1. Basic PSO (BPSO)

Basic PSO is an optimization algorithm inspired by the social behavior of fish and birds. The algorithm is simple, robust and efficient to find the optimal regions in the complex search spaces. PSO uses its members called a set of particles that represent the potential solutions that required to solve the optimization problem [38]. Each particle moves through its current velocity towards the optimal region and its position is updated at every iteration. PSO generates its global solution by comparing each particle solution.

Let N be the size of the population. Each particle is characterized by its position and velocity. The position vector of i th particle $x_i = (x_i^1, x_i^2, \dots, x_i^D)$ representing a co-ordinate in D -dimensional search space, where $i(1 \leq i \leq N)$ and velocity vector $v_i = (v_i^1, v_i^2, \dots, v_i^D)$ the velocity and position of each particle are updated using the following formula:

$$v_i^d(t+1) = v_i^d(t) + C_1 \cdot \phi_1 \cdot (pbest_i^d(t) - x_i^d(t)) + C_2 \cdot \phi_2 \cdot (gbest^d(t) - x_i^d(t)) \quad (1)$$

$$x_i^d(t+1) = x_i^d(t) + v_i^d(t+1) \quad (2)$$

where $x_i^d(t)$ and $v_i^d(t)$ represent d th component of the position and velocity of i th particle in iteration t , $C_1(C_1 \neq 0)$ and $C_2(C_2 \neq 0)$ are the acceleration constants, ϕ_1 and ϕ_2 are two random numbers in the range $[0,1]$. $pbest_i = \{pbest_i^1, pbest_i^2, \dots, pbest_i^D\}$ is the best position of the i th particle in iteration t , $gbest^d(t)$ is the global best position found so far in the entire swarm in iteration t .

The personal best position value of IPSO can be calculated as:

$$pbest_i(t+1) = \begin{cases} pbest_i(t), & \text{if } O_{bj}(x_i(t+1)) > O_{bj}(pbest_i(t)) \\ x_i(t+1), & \text{if } O_{bj}(x_i(t+1)) < O_{bj}(pbest_i(t)) \end{cases} \quad (3)$$

where O_{bj} denotes the particles fitness function and the global best position is obtained as:

$$gbest(t) = \min \{O_{bj}(pbest_1(t)), O_{bj}(pbest_2(t)), \dots, O_{bj}(pbest_N(t))\} \quad (4)$$

2.2. Improved particle swarm optimization

Population based optimization algorithms find their best solution in the search space by maintaining better control of exploration and exploitation characteristics. The basic PSO has hosted the thought of single leadership and single leadership does not always lead to generate an optimal solution in optimization algorithms, which invites premature convergence. Therefore, Basic PSO has been improved with the thought of governance in human society and tuned of parameters dynamically. In this improvement, PSO used two possible leaders who have been selected on the basis of the vote [39]. The opposition and leader have an equal number of impacts over the particles that have voted for them. The leader who leads the majority of particles whereas an opposition who leads for certain group of particles and always check the power of the leader that whether it is leading the society towards the optimum position. Again, inertia weight in PSO influences generating a better solution. If each particle updated its position and velocity with a greater value of the inertia factor, then it facilitates a global search, at the same time the lesser value facilitates a local search. Therefore, it is necessary to enhance the search ability of particles by adjusting the inertia weight dynamically. In this paper, the basic PSO has been improved with adaptive adjustment of weight, acceleration coefficients and concept of

governance in human society to enhance the convergence rate to best results in PSO and avoid premature convergence. The basic PSO has been revised and offered as follows:

$$\begin{aligned} v_i^d(t+1) = & w \cdot v_i^d(t) + C_1 \cdot \phi_1 \cdot (pbest_i^d(t) - x_i^d(t)) \cdot \left(1 - \frac{F}{F_{\max}}\right) \\ & + C_2 \cdot \phi_2 \cdot (lbest_i^d(t) - x_i^d(t)) \cdot \left(1 - \frac{F}{F_{\max}}\right) \\ & + C_3 \cdot \phi_3 \cdot (Leader_i^d(t) - x_i^d(t)) \cdot e^{(1-vote_{Leader})} \end{aligned} \quad (5)$$

$$x_i^d(t+1) = x_i^d(t) + C_4 \cdot \phi_4 \cdot v_i^d(t+1) \quad (6)$$

The velocity update Eq. (5) consists of four terms, the first term inertia weight(w) is multiplied with velocity term at iteration t . Several mechanisms have been employed to enhance the performance of the PSO by fine tuning its parameter. These changes have been made keeping the basic organization of the algorithm unchanged. The velocity at next time magnified by scaling with a scalar factor (w), the inertia weight. The global search will be attained when the value of the inertia weight is large and the local search is attained when the inertia weight value is small. The ability of search is improved by varying inertia weight dynamically. The adaptive method of turning the inertial weight is projected [40,41] and offered in Eq. (7). Empirical experiments have been achieved in the earlier with an inertia weight that differs in the range [0.9, 0.4].

$$w = \frac{1}{1 + 1.5e^{-2.6s}} \quad (7)$$

where s is the evolutionary factor as defined in the Eq. (8). Here, the initial value of inertia weight is 0.4 and the final value of the inertia weight is 0.9.

$$s = \frac{dis_g - dis_{\min}}{dis_{\max} - dis_{\min}} \in [0, 1] \quad (8)$$

$$dis_i = \frac{1}{N-1} \sum_{j=1, j \neq i}^N \sqrt{\sum_{k=1}^D (x_i^k - x_j^k)^2} \quad (9)$$

Where dis_i is the mean distance of particle i to all other particles using Euclidian metric, N is the population size and D is the number of dimensions. dis_i is the global best particle and presented as dis_g and also comparing all dis_i 's, maximum ($dis_{\max}$) and minimum ($dis_{\min}$) distance will be determined and $dis_i \in [0, 1]$. Similarly, the constant value is assigned to the acceleration coefficients (usually constant to 2.0). Particle leads to a local optimum when the social component value C_2 is greater than the cognitive component C_1 and comparatively large value of C_1 resulting the particles to roam around the solution domain. The linear variation of C_1 and C_2 with iteration improved the solution quality. The coefficients are updated through the following Eqs. (10) and (11).

$$C_1 = C_{1i} - \left(\frac{C_{1i} - C_{1f}}{Max_Iter} \right) k \quad (10)$$

$$C_2 = C_{2i} + \left(\frac{C_{2f} - C_{2i}}{Max_Iter} \right) k \quad (11)$$

Where, initial and final values of acceleration coefficients are C_{1i} , C_{1f} , C_{2i} and C_{2f} and Max_Iter is the highest number of iterations. Here, C_3 and C_4 are attraction coefficient and velocity scaling factor respectively. In our experiment, the value of $C_3 = 0.4$ and $C_4 = 0.7$. The acceleration coefficient and inertial weights are the influential parameters to enhance the solution accuracy [42–44]. The ability of the algorithm has enhanced by combining other search techniques with PSO [45]. Combination of evolutionary

operators with PSO enhances the population diversity to avoid from neighborhood minima [46–48] such as selection crossover and mutation. Therefore, the subsequent section describes the use of two evolutionary operators in enhancing the performance of PSO.

The second term represents the difference between the current position of the particle $x_i^d(t)$ and personal best position $pbest_i^d(t)$ multiplied with another factor $(1 - \frac{F}{F_{\max}})$ to ensure faster convergence and expected a particle to be reached its optimum position after each iteration. The term $\frac{F}{F_{\max}}$ represents the ratio of the function evaluations. The distance between $pbest_i^d(t)$ and $x_i^d(t)$ is anticipated to decline and hence, this part unable to improve its velocity. The third term is based on the local best, where individual particle examines its peer environment and find the superior one among them. The difference between $lbest_i^d(t)$ and $x_i^d(t)$ is calculated and multiply with other factors C_2 , ϕ_2 and $(1 - \frac{F}{F_{\max}})$. This also ensures the faster convergence and the impact of this term decreases slowly. The term $lbest_i$ in Eq. (5) represents the local best position and the value each particle determined as per the following expression

$$lbest_i = \min \{O_{bj}(lbest_{i+1}(t)), O_{bj}(lbest_{i+2}(t)), \dots, O_{bj}(lbest_N(t))\} \quad (12)$$

The fourth term is based on the voting concept where the particles will be enticed to their voted leader and move towards it. As the iteration proceeds the opposition vote ($opVote$) decreases and the governance vote ($goVote$) increases due to the better leadership in the governance. Hence, the influence of opposition leads become weak and provide the faster convergence, but it can be hindered if the leader in the governance gets damped at local minima and opposition leads the particles to global optima. The effect of the opposition will be reinforced if $opBest$ is very small. Therefore, the factor $e^{(1-vote_{Leader})}$ is multiplied empirically to avoid the above situation. The voting process is calculated by using the roulette wheel mechanism. The asymmetric ranges considered for $goVote$ and $opVote$ are kept in range [0,1] by adopting the normalization procedure. The initial biasing value allows the population to be created in a random manner and to decide their own leader. Initially, the particle with better fitness value is treated as governor and the particle with the second best fitness values as an opposition. If φ is the value for $goVote$, then $1 - \varphi$ is the value for $opVote$. This initial value φ bias the vote. In each subsequent iterations, every particle poll a vote($vote_i^d$) which is a number selected randomly within [0,1] and leader is nominated as per the following manner:

$$Leader_i^d = \begin{cases} Governor^d, & vote_i^d \leq goVote \\ opposition^d, & vote_i^d > goVote \end{cases} \quad (13)$$

$goVote$ and $opVote$ are updated after selection of the leader and the vote count is updated as follow:

if $vote_i^d < goVote$ then

$$goVote = goVote + \frac{1}{M * N} \quad (14)$$

Else

$$opVote = opVote + \frac{1}{M * N} \quad (15)$$

The term M represents the number of votes cast in the favor of leader in a given dimension of the problem of interest. The cumulative format of votes is considered of total votes balloted in the favor of any candidate. These cumulative votes indicate which leader always acts as a better leader. The votes acquired

at the completion of every iteration is normalized through the following Equation and carry forward for next iteration.

$$goVote = \frac{goVote}{goVote + opVote} \quad (16)$$

$$opVote = \frac{opVote}{goVote + opVote} \quad (17)$$

$$\text{if } goVote = \varphi \text{ then} \\ opVote = 1 - \varphi \quad (18)$$

$vote_{Leader}$ is the standardized number of votes expected by a specific leader. The emphasize of empirical expression $e^{(1-vote_{Leader})}$ is analyzed as follow:

Case 1: $Leader_i^d = Governor_d$, in this case $vote_{Leader} = goVote$ which nearly inclines to 1.

Case 2: $Leader_i^d = opposition_d$, in this case $vote_{Leader} = opVote$ and $opVote$ is very lesser then $e^{(1-opVote)} \approx e^1 \approx 2.3$.

The $Governor_d$ and $opposition_d$ are updated through the following equations:

$$opposition_d = \begin{cases} \phi \times (X_{\max} - X_{\min}) + X_{\min}, & \text{if } rand_d < pro \\ Governor_d, & \text{if } rand_d \geq pro \end{cases} \quad (19)$$

$$Governor_d = gbest_d \quad (20)$$

where ϕ and $rand_d$ are random number within $[0,1]$, $pro = \frac{1}{N}$ is taken empirically, where $d \in \{1, 2, 3 \dots D\}$ and $opposition_d$ is set to a random number distributed uniformly in the interval $[X_{\max}, X_{\min}]$ otherwise $opposition_d$ is taken from the best particle.

3. Proposed evolutionary operators

Two operators are introduced in this section, the first operator is based on multi-crossover [49] and the second one is similar to bee colony techniques [50]. These two operators help to improve the convergence and escape from local minima.

Operator 1: Operator 1 is based on multi-crossover in generating algorithm [49]. The multi-crossover used three chromosomes $(\alpha_1, \alpha_2, \alpha_3)$ which has been selected randomly from mating pool, if the fitness value of the chromosome α_i (where, $i \in [1, 2, 3]$) is smallest, it will be selected as the premier parent and the new chromosome α'_i is generated in Eq. (21).

$$\alpha'_i = \alpha_i + rand(2\alpha_1 - \alpha_2 - \alpha_3) \quad (21)$$

where $rand$ is a random value in the closed interval of 0 and 1. Here, these operator employs the premier parent (x_{gbest}) and second parent (x_{pbest}) for a crossover game. The particle new velocity will generate using the following Equation:

$$V_i^d(t+1) = rand(\frac{C_2}{2} x_{gbest}^d(t) - x_i^d(t) - x_{pbest}^d(t)) \quad (22)$$

Here, C_2 is the social learning factor, x_{gbest} is the global best position, x_{pbest} is the local best position and $rand \in [0,1]$ is the random value.

Operator 2: This operator uses the concepts of the Artificial Bee Colony algorithm(ABC) [51], in which employed, scout bees and onlookers are replicated. Bees are classified as per their food source. They are found with food source are termed as Employed bees where bees whose food source are unrestricted become a scout. Onlooker bees, on the other hand, decide the food source using the information shared by the employed bees. This algorithm runs in five phases, in the first phase, initially, food sources are random. In the second phase, the distribution of employed bees modifies her location in the food source sites based on the local information and locates a nearby food source. In this respect, the replacement of the old position with a new one is decided, based on the improvement of the fitness value.

The third phase involves the determination of the probability value based on the evaluation of the fitness values. The onlooker bee selects the food source through the information delivered by the employed bee is a process in the fourth phase; and in the final phase, the abandonment is checked [50].

Here, the particles are selected in the PSO strategy through the finding of the food source operator in the ABC algorithm. The position of the particle in d th dimension is updated as per the following expression

$$x_i^d(t+1) = x_i^d(t) + (2rand - 1)(x_i^d(t) - x_j^d(t)) \quad (23)$$

Here, $rand$ is the random value between $[0,1]$, d is the integer random number in the range $[1, \text{dimension}]$ and j is an integer random number in the range $[1, \text{no of particles}]$. The better value should be selected between $x_i^d(t+1)$ and $x_i^d(t)$ after computation of Eq. (23)

4. Hybrid of IPSO and evolutionary operators(EOPs)

The classical PSO algorithm suffers to generate an optimal solution and a slower convergence rate due to the dependency on external parameters like inertia weight and acceleration parameters. Therefore, the efficiency of the PSO algorithm is comparatively poor. Since the above problem encounters in the PSO algorithm, some measures have been taken care of to improve the performance of PSO as presented in the previous section. But experimentally, it has been observed that in most cases the convergence is premature yielding a fewer optimal solution when most of the particles do not change their position in the swarm in the successive stages, so the global optimum cannot be discovered [52]. This situation occurs due to the lesser value of the constriction coefficient or inertia weight. In Eq. (5), it has been obtained that, if V_i^d , $|pbest_i^d - x_i^d|$, $|Leader_i^d(t) - x_i^d(t)|$ and $|lbest^d - x_i^d|$ contain less value then V_i^d cannot achieve an enormous value in the forthcoming generation. That shows a defeat of intensification ability occurs at the primary phase of the exploration process. The global best position of particle instigates $|pbest_i^d - x_i^d|$, $|Leader_i^d(t) - x_i^d(t)|$ and $|lbest^d - x_i^d|$ to be zero and becomes inhibited at rapidly with the ratio w . It also suffers from loss of the diversity, when $pbest(t)$, $Leader_i^d(t)$ and $lbest(t)$ are close enough [16,1]. Due to the problems encountered above, the evolutionary operator is inured to regulate the velocity of the IPSO to eradicate premature convergence.

In this hybridization, the particle positions of the improved PSO are updated using the two proposed evolutionary operators. Initially, the population is generated randomly and the inertia weight, learning factor, and fitness values of all particles are calculated in each iteration. Then, the x_{pbest}^d and x_{gbest}^d is determined by the fitness values of all particles. Two random numbers $\lambda_1, \lambda_2 \in [0,1]$ is assigned to every particle. The bee colony operator presented in Eq. (23) is used produced to a new particle, if the particle does not have $\lambda_1 > (fitness \text{ values based on standard deviation})$ or $\lambda_2 < \frac{P_B \times t}{\max_iteration}$, where the term P_B represents the probability calculated in a bee colony, t refers to the present iteration and maximum allowable iterations is represented by $\max_iteration$. The particle is assigned with random number $\delta \in [0,1]$ when it is not selected from the past operation. The velocity of the new particle will be generated using multi-crossover operator by Eq. (22), if a particle has $\delta < P_c$, where P_c is the multi-crossover probability. The particles that are not selected from the multi-crossover or bee colony operation will be enhanced by IPSO of Eq. (5) and Eq. (6). This process will be continuing until the stopping condition is reached. In the end, x_{gbest} would be calculated as the prominent solution.

The relative performance of our proposed algorithm has been tested with other competitor algorithms using 10 benchmark

Algorithm IPSO-EOPs (x^{curr-i}, y^{curr-i})

Initialize: $\bar{X} = (x^{curr-i}, y^{curr-i})$ and Velocity V_i randomly for total population.
 $goVote := \phi \rightarrow 0.7$, $opVote = 1 - \phi \rightarrow 0.3$. update $pbest$ and $lbest$. Generate *Governer* and *opposition*

1. **While** ($F < F_{\max}$)
2. **For** $t = 1$ to $\max_iter$
3. **For** $i = 1$ to N
4. **For** $d = 1$ to D
5. **If** ($F(\bar{X}_i(t)) \leq F(\bar{X}_{pbest_i}(t))$)
6. $\bar{X}_{pbest_i}(t) = \bar{X}_i(t)$
7. **Else**
8. $\bar{X}_{pbest_i}(t) = \bar{X}_{pbest_i}(t-1)$
9. **End if**
10. **If** ($F(\bar{X}_i(t)) \leq F(\bar{X}_{lbest_i}(t))$)
11. $\bar{X}_{lbest_i}(t) = \bar{X}_i(t)$
12. **Else**
13. $\bar{X}_{lbest_i}(t) = \bar{X}_{lbest_i}(t-1)$
14. **End if**
15. **If** ($Vote_i^d < goVote$)
16. $Leader_i^d \leftarrow Governor^d$, $goVote = goVote + \frac{1}{N+M}$, $Vote_{Leader} \leftarrow goVote$
17. **Else**
18. $Leader_i^d \leftarrow opposition^d$, $opVote = opVote + \frac{1}{N+M}$, $Vote_{Leader} \leftarrow opVote$
19. **End if**
20. $F(\bar{X}_{pbest_i}(t)) = \min F(\bar{X}_{pbest_i}(t))$
21. **If** ($(rand > \text{standard deviation}(F_{std})) \text{ or } rand < \frac{PB \times t}{\max_iteration}$)
22. **If** ($rand < P_C$)
23.
$$v_i^d(t+1) = w.v_i^d(t) + C_1.\phi_1.(pbest_i^d(t) - x_i^d(t)).(1 - \frac{F}{F_{\max}})$$

$$+ C_2.\phi_2.(lbest_i^d(t) - x_i^d(t)).(1 - \frac{F}{F_{\max}}) + C_3.\phi_3.(Leader_i^d(t) - x_i^d(t)).e^{(1-vote_{Leader})}$$
24. **Else**

```

25.       $v_i^d(t+1) = rand(\frac{C_2}{2} leader_i^d - X_i^d(t) - lbest_i^d(t) - pbest_i^d(t))$ 
26.      End if
27.      Else
28.       $X_i^d = X_i^d + (2rand - 1)(X_i^d - X_i^d)$ 
29.       $X_{new}^d = X_i^d, X_{new}^d = X_i^d$ 
30.      If  $F(\bar{X}_{new}) < F(\bar{X}_i)$ 
31.       $X_i^d = X_{new}^d$ 
32.      End if
33.      End if
34.      End for
35.      If  $(F(\bar{X}_i) < F(pbest_i))$ 
36.       $pbest_i = \bar{X}_i$ 
37.      Else if  $(F(\bar{X}_i) < F(lbest_i))$ 
38.       $lbest_i = \bar{X}_i$ 
39.      Else if  $(F(\bar{X}_i) < F(lbest_{i \pm 1}) \text{ or } F(\bar{X}_i) < F(lbest_{i \pm 2}))$ 
40.       $lbest_{i \pm 1} = \bar{X}_i$ 
41.      Else
42.      Update opposition and Governor using Eq. 19 and Eq. 20
43.      If End
44.      For End
45.      For End
46.      Normalize goVote and opVote
47.      End while

```

Table 1
Benchmark functions.

Functions	Name	Range	Optima	Dimension
f_1	Sphere	$[-100, 100]$	0	30
f_2	Rosenbrock	$[-30, 30]$	0	30
f_3	Rastrigin	$[-5.12, 5.12]$	0	30
f_4	Ackley	$[-32, 32]$	0	30
f_5	Griewank	$[-600, 600]$	0	30
f_6	Schwefel 2.21	$[-500, 500]$	-12,569.5	30
f_7	Shifted Sphere	$[-100, 100]$	-450	30
f_8	Shifted RotatedRastrigin's	$[-5, 5]$	-330	30
f_9	Shifted RotatedWeierstrass	$[-0.5, 0.5]$	90	30
f_{10}	Shifted Rosenbrock's	$[-100, 100]$	390	30

functions of dimension 30. The list of benchmark functions [33] considered for varying the performance has been presented in Table 1. The experiments are conducted for 30 independent runs and 100 iterations per run. The maximum generation and population size are set to 50. Furthermore, to verify the performance of our proposed algorithm, we made a comparison with other

existing variants of PSO such as Comprehensive Learning Particle Swarm Optimization (CLPSO) [45], Orthogonal Learning Particle Swarm Optimization (OLPSO) [41]. The initial parameters for all competitor algorithms are set the same as the proposed algorithm and $m = 8$, set for CLPSO. The benchmark functions have been tested and presented in Table 2. The statistical significance of the

Table 2
Comparative analysis of Performance of IPSO-EOPs with other algorithms based solution quality for $f_1 - f_{10}$.

Algorithm	IPSO-EOPs			IPSO			Stat. significant	CLPSO			Stat. significant	OLPSO			Stat. significant
	Mean	Best	Std. Dev.	Mean	Best	Std. Dev.		Mean	Best	Std. Dev.		Mean	Best	Std. Dev.	
f_1	0.00	0.00	0.00	1.729e-153	1.84e-163	7.6e-160	E+	1.42e-25	2.176e-28	2.754e-27	E+	1.57e-13	5.28e-15	8.3e-14	E+
f_2	27.044	26.549	0.2824	6.565	3.623e-7	6.243	E+	16.312	9.265e-1	12.453	E+	11.47	8.68e-7	8.29	E+
f_3	0.00	0.00	0.00	2.658e-13	6.125e-15	1.746e-13	E+	2.46e-14	0	4.389e-16	E+	9.03e-06	4.97e-17	1.35e-04	E+
f_4	8.88e-17	8.88e-17	0.00	1.795e-15	6.052e-15	2.638e-15	E-	2.538e-16	1.937e-16	4.394e-16	E-	4.08e-08	1.67e-10	7.93e-09	E-
f_5	0.00	0.00	0.00	1.277e-02	0.00	1.527e-03	E+	1.697e-15	0.00	8.174e-15	E+	9.18e-10	0.00	8.61e-10	E+
f_6	0.00	0.00	0.00	1.478e-87	1.649e-94	4.395e-91	E+	3.461e-18	1.139e-18	1.759e-18	E-	5.48e-09	1.59e-09	5.734e-10	E+
f_7	0.00	0.00	0.00	0.00	0.00	0.00	E+	0.00	0.00	0.00	E+	0.00	0.00	0.00	E+
f_8	113.45	78.78	23.294	112.633	61.395	27.956	E+	101.943	77.967	16.730	E+	167.35	44.564	27.83	E-
f_9	22.674	19.472	3.83	22.352	15.335	3.728	E+	24.574	20.057	1.735	E-	24.65	20.547	3.345	NA
f_{10}	27.56	21.62	1.32	23.216	4.63e-05	42.89	E+	14.78	4.39e-02	21.583	E+	3.673	23.48	12.43	E+

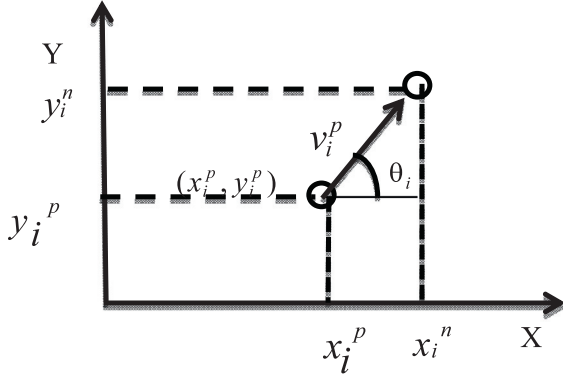


Fig. 2. Illustration of next position from current position for the i th robot.

best two algorithms by taking the difference of mean, obtained using paired t-test with 10 samples, is presented in the last column of Table 2. Here 'E+' indicates that IPSO-EOPs is significantly better than its competitor algorithm at 0.05 levels of significance by two-tailed test, 'E-' indicates that the difference of the mean is not statistically significant and 'NA' stands for not applicable.

5. Formulation of problem for path planning of multi-robot

The main aim of the multi-robot path planning problem is to generate a collision free path for each robot from their current position to target position in a cluttered environment. Before formulating the problem, some assumption has been considered such as each robot in the environment knows their initial position and target position. During the movement of the robot, it selects any action from their existing action to reach the next subsequent position and the process will continue until the arrival of their target position. Initially, heading direction of the robot is aligned towards their target position and during the moment, if any obstacle arrives in its path, then, the robot has to re-select their action (to turn any angle) for avoiding the obstacle. The objective of the problem formulation is to generate a fitness function that minimizes path length for every robot, arrival time, optimizing the energy in the form of the number of rotations, avoiding the static and dynamic obstacles and smoothness of the path. The fitness function will be used for determining the subsequent collision free position that satisfies all the constraints mentioned above and the process will be iterated till of each robot reach at their target position.

Consider the initial position, goal position and current velocity of the robot R_i at time t is (x_i^p, y_i^p) , (x_i^g, y_i^g) and v_i^p respectively. (x_i^n, y_i^n) is next position when robot R_i is decided to move in time $(t + \Delta t)$.

So, the expression for the next position (x_i^n, y_i^n) can be expressed from Fig. 2 as follow

$$x_i^n = x_i^p + v_i^p \cos \theta_i \Delta t \quad (24)$$

$$y_i^n = y_i^p + v_i^p \sin \theta_i \Delta t \quad (25)$$

Putting $\Delta t = 1$, the Eq. (24) and Eq. (25) is rewrite as:

$$x_i^n = x_i^p + v_i^p \cos \theta_i \quad (26)$$

$$y_i^n = y_i^p + v_i^p \sin \theta_i \quad (27)$$

The fitness function F_1 is used to determine the trajectory of path length for n^R number of robots,

$$F_1 = \sum_{i=1}^{n^R} \sqrt{(x_i^p - x_i^n)^2 + (y_i^p - y_i^n)^2}$$

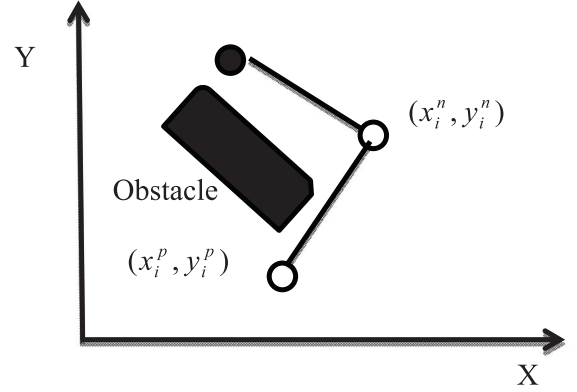


Fig. 3. Selection of next position (x_i^n, y_i^n) from the current position (x_i^p, y_i^p) for avoiding collision with the obstacle.

$$+ \sqrt{((x_i^n - x_i^g)^2 + (y_i^n - y_i^g)^2)} \quad (28)$$

By putting the value x_i^n and y_i^n from the Eq. (26) and Eq. (27) into the Eq. (28), we find,

$$F_1 = \sum_{i=1}^{n^R} \left\{ v_i^p + \sqrt{(x_i^p + v_i^p \cos \theta_i - x_i^g)^2 + (y_i^p + v_i^p \sin \theta_i - y_i^g)^2} \right\} \quad (29)$$

The objective function formulated in Eq. (28) is to minimize the distance between the initial position to target position for n^R number robots. Now, path planning of multiple mobile robots is formulated as an optimization problem by minimizing the objective function in Eq. (28) assuming the set of constraints present in the environments are as follows: (1) robot should not collide with its teammates; (2) robot should not make any collision with static or dynamic obstacles. The second objective function is formulated to avoid static obstacles present in the environment. The objective function formulated in Eq. (30) is to escape the collision between static obstacles and robots (see Fig. 3).

$$Ob_j = \begin{cases} 1 & \text{if } dist(O_j) \leq dist(O_j)_{min} \\ \exp(-\beta \cdot \frac{dist(O_j) - dist(O_j)_{min}}{dist(O_j)_{max} - dist(O_j)_{min}}) & \text{if } dist(O_j)_{min} < dist(O_j) < dist(O_j)_{max} \\ 0 & \text{if } dist(O_j) \geq dist(O_j)_{max} \end{cases} \quad (30)$$

where β is a positive constant, $dist(O_j)$ be the distance between obstacles and robot, $dist(O_j)_{max}$ is maximum distance and $dist(O_j)_{min}$ is the minimum distance with respect to the obstacle O_j . The safe and collision path is obtained, when $dist(O_j) \geq dist(O_j)_{max}$ and path is unsafe if, $dist(O_j) \leq dist(O_j)_{min}$. Thus, mathematically, the optimization problem for ' N ' number of static obstacles can be expressed as follows:

$$F_2 = \max_{j=1,2,\dots,N} (ob_j) \quad (31)$$

The third function is considered to avoid collision between teammates and it is expressed mathematically as follows:

$$F_3 = \sum_{j=1}^m \sum_{k=1}^m \frac{1}{D(j, k)} \quad (32)$$

The minimum distance between j th robot and k th robot, $D(j, k)$ must be greater than $2r$, where r is the radius of the robot. The fourth objective function is designed for predicting dynamic obstacle position. The objective function is expressed as below:

$$F_4 = \sum_{i=1}^{n^R} \sqrt{(x_p - x_i^g)^2 + (y_p - y_i^g)^2} \quad (33)$$

where (x_p, y_p) is the projected place of the dynamic obstacles. Now $x_p = x_i^{obs} + u\delta t$ and $y_p = y_i^{obs} + u\delta t$ where δt is the time where $\delta t = ar[i]/ar[j]$, $ar[i]$ stores the entire time required for movement of dynamic obstacle and $ar[j]$ stores the number of steps necessary for the dynamic obstacle. (x_i^{obs}, y_i^{obs}) existing position of the dynamic obstacle. Here, u is the velocity of the dynamic obstacle. The projected prepared at this time is linear because extrapolation of the path of the dynamic obstacle is made considering the direction of motion constant over given interval of time

Again, the fifth objective function is considered to express the smoothness of the path. The smoothness of the path is articulated as the angle between two imaginary lines connecting the goal point and two consecutive locations of the robot's in every iteration, i.e. $gbest^i$ and $gbest^{i-1}$ in i th iteration. Mathematically, the fitness function formulated for the smoothness of path is as follows:

$$F_5 = \frac{\cos^{-1} \left[\frac{(x_i^p - x_i^g) \cdot (x_{gbest^{i-1}} - x_i^g) + (y_i^p - y_i^g) \cdot (y_{gbest^{i-1}} - y_i^g)}{\sqrt{(x_i^p - x_i^g)^2 + (y_i^p - y_i^g)^2} \sqrt{(x_{gbest^{i-1}} - x_i^g)^2 + (y_{gbest^{i-1}} - y_i^g)^2}} \right]}{1} \quad (34)$$

At present, the multiple mobile robot path planning problem of mobile robots can be presented as an optimization problem where both path and energy consumption are optimized. The optimization problem is formulated as constraints-based multi-objective functions. It consists of an objective function with four constraints. The objective function represents the minimization of the path length between the existing position of each robot and its destination. The constraint is formulated to escape collision with colleagues, static and dynamic obstacles and generate a smoothness of the path. It has been modeled as four types of penalty functions. The mobile robots avoid the collision with static obstacles by first penalty function, the second function is designed to evade the collision among the teammates. At the same time, it is also avoiding collision with dynamic obstacles by the third penalty function and the smoothness path is reflected in the fourth penalty function. Again the multi-objective function has been formulated as a single objective function of the weighted sum of four objective functions. Therefore, the overall objective (or fitness) functions have been expressed as follows:

$$F = \lambda_1 F_1 + \lambda_2 F_2 + \lambda_3 F_3 + \lambda_4 F_4 + \lambda_5 F_5 \quad (35)$$

Where $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ and λ_5 represents the weights of the shortest path, non-moveable obstacles, teammates, moveable obstacles and smoothness of path respectively. These weights are tuned in the simulation and real platform environment, with foremost values found $\lambda_1 = 1, \lambda_2 = 1, \lambda_3 = 1, \lambda_4 = 0.5$ and $\lambda_5 = 0.5$. Eq. (35) is employed to obtain the optimal path by minimizing the fitness function.

5.1. Average total trajectory path deviation (ATTPD)

Let TP_{kj} be a collision free path computed by the program from the initial point S_k to the target point G_k for robot R_k in the j th run. If $TP_{k1}, TP_{k2}, \dots, TP_{kj}$ are the collision free paths computed over j runs, then the average total trajectory path traversed (ATTPT) of robot R_k is given by $\sum_{r=1}^j TP_{kr}/j$. The average trajectory path deviation for robot R_k is expressed as the difference between the ideal shortest path of R_k from source S_k to target G_k and ATTPT of R_k . If the ideal trajectory path for robot R_k obtained geometrically is TP_{k-real} , then the total average trajectory path deviation is expressed by

$$ATTPD = TP_{k-real} - \sum_{r=1}^j TP_{kr}/j \quad (36)$$

Therefore, in multi-robot environment the average total trajectory path deviation (ATTPD) for n robots is expressed as follows:

$$ATTPD = \sum_{k=1}^n (TP_{k-real} - \sum_{r=1}^j TP_{kr}/j) \quad (37)$$

5.2. Average untraveled trajectory target distance (AUTTD)

Consider G_k and C_k are goal and current position of a robot R_k on a 2-D Environment, where G_k and C_k are 2-dimensional vectors, the uncovered (untraveled) trajectory distance for robot k is $\|G_k - C_k\|$, where $\|\cdot\|$ denotes Euclidean norm.

For n robots, uncovered trajectory target distance (UTTD) is the sum of $\|G_k - C_k\|$,

$$UTTD = \sum_{i=1}^n \|G_k - C_k\|. \quad (38)$$

Now, for j th runs of the program, we evaluate the average of UTTDs, and call it the average uncovered target distance (AUTTD) as :

$$AUTTD = \sum_{i=1}^n \|G_k - C_k\|/j. \quad (39)$$

6. Hybrid IPSO-EOPs algorithm for path planning of multi-robot

In this section, the path planning of multiple mobile robots has been resolved through the projected algorithm. The single objective function presented in Eq. (35) is used in the IPSO-EOPs for computing the successive optimal position of every robot from their existing position at the same time satisfying all the constraints and enhanced the convergence speed. The proposed algorithm also generates the optimal path for every robot from their initial position to the designation in a static and dynamic environment. A robot takes the help of its fortified sensors to evaluate the distance from obstacles. The outline of the algorithm for path planning of multiple mobile robots is discussed in **Algorithm of path planning**.

7. Computer simulation

The simulated environment has been considered for realizing the mobile robots path planning problem. The simulation has been carried out through C programming language. Each robot in the simulation is designated as a circular shape of radius 6 pixels with different color codes. Before the beginning of the experiment; the specified initial position and goal position is assigned for n^R number of robots. The obstacles have been placed randomly on the world map. While performing the experiments, two cases are considered; in case 1; both obstacles and environments are motionless comparative to a robot; while other robots are moveable for priority robots; in case 2; both obstacles and robots are moveable. Equal velocities are assigned to each and every robot during the start of the experiment through the execution of the program; however, the velocities are tuned during the run of the program and program will be repeated for a certain number of steps, or until some suitable solution has been found by the algorithm. After completion of the number of iterations, most of the particles are expected to converge to the global optima of the search space. The world map of size 450×400 is considered for exercising the path planning of multi-robot.

The configuration of the world map for exercising the experiment has been presented in Fig. 4 with seven obstacles and six

Algorithm of path planning

Input: v_i^p , (x_i^p, y_i^p) and (x_i^g, y_i^g) are the velocity, initial position and goal position for n^R robots

respectively, $1 \leq i \leq n^R$ and the threshold value ε

Output: Optimal Trajectory of path OTP_i is generated for the robot R_i from (x_i^p, y_i^p) to (x_i^g, y_i^g)

Begin

For $i = 1$ to n^R

$x_{curr_i}^p \leftarrow x_i^p$; $y_{curr_i}^p \leftarrow y_i^p$;

End for

For every robot $i = 1$ to n^R

While ($Curr_i \neq G_i$) // $curr_i = (x_i^p, y_i^p)$, $G_i = (x_i^g, y_i^g)$ //

Call IPSO-EOPs ($x_{curr_i}^p, y_{curr_i}^p$);

Moveto ($x_{curr_i}^p, y_{curr_i}^p$);

End for

End for

End

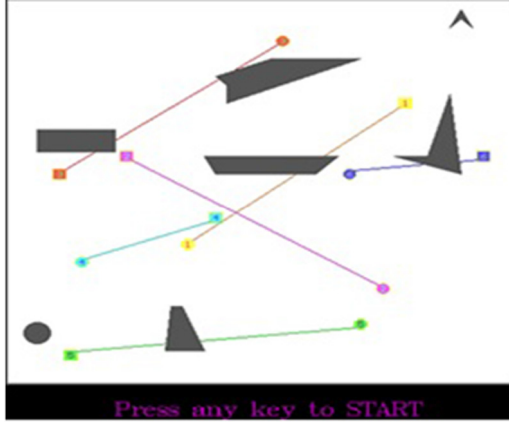


Fig. 4. Initial Configuration of static Environment.

robots with different colors. The circular shape represents the robot and the rectangle shape signifies the target position of every robot in a similar color. The simulation has been exercised to decide the initial velocity for each robot and the performance of mobile robots has been evaluated by fluctuating robots number with velocity. The initial velocity is decided based on the performance evaluated in Table 3. The following parameters have been tuned and set in the simulation and real platform: No_of_particles = 50, dimension = 2, $P_B = 0.03$ and $P_C = 0.95$. In IPSO-EOPs, C_1 is linearly reduced from $C_{1i} = 2.5$ to $C_{1f} = 0.5$ and C_2 is increased linearly from $C_{2i} = 0.5$ to $C_{2f} = 2.5$. The parameters are chosen based on their best value computed through the simulation process. The influence of the parameters of IPSO-EOPs is decided based on Table 4. The control parameter of IPSO is considered from the parameters of IPSO-EOPs and the ABPSO has no control parameter to be tuned. The program is executed 30 times and the average value of the result is considered for evaluating the performance.

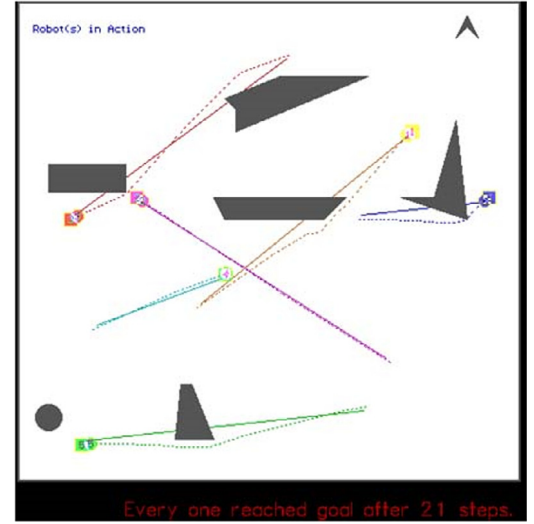


Fig. 5. Six robots arrived at their corresponding goal position after 21 steps through IPSO-EOPs in the static environment.

Each robot decides its next position based on the fitness value defined in Eq. (35) through the run of the program. Fig. 5 indicates that every robot reached in their corresponding target position in 21 steps using IPSO-EOPs. The dotted line represents the trajectory path for every robot in the same color code. The trajectory path generated through IPSO and DE is offered in Fig. 6 and Fig. 7 respectively. Every robot arrived at their corresponding target within 25 and 31 steps correspondingly.

Similarly, the simulation has been exercised with 6 robots and 10 obstacles, where the obstacles and robots are dynamic in nature presented in Fig. 9. Intermediate steps of movement of robots have been presented in Fig. 10, where robot number 1 is re-computing its path due to the moveable obstacles. Similarly, after a few steps of movement again robot number 1 and robot number 2 are replanning (re-compute) their path due to dynamic obstacles which are illustrated in Fig. 11. Every robot arrived at

Table 3
Effect of velocity on the performance evaluation.

Velocity (Pixels/s)	Static environment				Dynamic environment		
	Number of robots	Steps Required	average untraveled target distance (In Pixels)	Run time (In s)	Steps required	Average untraveled target distance (In Pixels)	Run time (In s)
10	2	14	135.37	31.56	17	148.90	34.72
	3	17	147.25	37.45	21	162.27	41.20
	4	21	152.78	43.27	24	183.34	47.60
	5	23	187.49	49.18	27	211.86	55.08
	6	26	248.28	54.33	31	278.07	59.76
12	2	12	187.32	27.23	15	191.06	28.86
	3	14	217.24	33.29	19	223.75	35.28
	4	18	237.89	41.36	21	240.75	43.84
	5	20	249.28	47.42	23	254.27	50.26
	6	21	253.34	52.45	26	257.09	55.60
14	3	16	223.19	38.76	21	253.32	42.64
	2	13	217.76	32.67	18	248.24	36.92
	4	20	237.39	44.23	23	272.53	48.65
	5	22	243.67	51.09	25	277.30	56.20
	6	27	257.45	53.71	29	295.56	60.16
16	2	14	227.45	30.17	16	257.02	32.94
	3	16	233.45	39.06	22	266.13	43.74
	4	19	251.34	43.80	26	289.04	49.05
	5	23	257.89	50.35	29	295.28	55.38
	6	25	265.35	54.92	33	305.16	60.41

Table 4
Tune of parameters for evaluating the performance of mobile robots path planning.

Parameters	Required steps	Required turn	ATTPD (in pixels)	AUTTD	Execution time (in s)	Other parameters
$P_B = 0.01$	26	19	268.45	291.78	65.23	$C_{1i} = 2.5$,
$P_B = 0.02$	23	17	248.75	262.45	58.65	$C_{2i} = 0.5$,
$P_B = 0.03$	21	14	216.67	253.34	52.45	$P_C = 0.95$
$P_B = 0.04$	25	20	252.34	278.56	57.89	$C_{1f} = 0.5$
$P_B = 0.05$	27	22	275.68	289.67	67.74	$C_{2f} = 2.5$
$C_{1i} = 1$	27	21	272.58	267.45	64.69	$C_{2i} = 0.5$,
$C_{1i} = 1.5$	25	19	262.93	265.89	60.28	$C_{1f} = 0.5$,
$C_{1i} = 2$	23	17	245.42	257.63	55.25	$C_{2f} = 2.5$,
$C_{1i} = 2.5$	21	14	216.67	253.34	52.45	$P_C = 0.95$,
$C_{1i} = 2.7$	29	22	269.83	259.23	65.98	$P_B = 0.03$
$C_{1f} = 0.2$	31	22	264.68	269.67	66.45	$C_{2i} = 0.5$,
$C_{1f} = 0.3$	29	19	252.38	262.47	61.89	$C_{1i} = 2.5$,
$C_{1f} = 0.4$	25	17	245.32	256.98	56.76	$C_{2f} = 2.5$,
$C_{1f} = 0.5$	21	14	216.67	253.34	52.45	$P_C = 0.95$,
$C_{1f} = 0.6$	27	21	257.89	259.76	67.34	$P_B = 0.03$
$C_{2i} = 0.2$	27	21	275.56	263.78	68.45	$C_{1f} = 0.5$,
$C_{2i} = 0.3$	26	19	257.23	261.45	63.25	$C_{1i} = 2.5$,
$C_{2i} = 0.4$	23	17	248.32	257.87	58.24	$C_{2f} = 2.5$,
$C_{2i} = 0.5$	21	14	216.67	253.34	52.45	$P_C = 0.95$,
$C_{2i} = 0.6$	23	16	226.78			$P_B = 0.03$
$C_{2f} = 1$	29	23	267.89	267.89	67.17	$C_{1f} = 0.5$,
$C_{2f} = 1.5$	27	21	243.23	263.46	63.45	$C_{1i} = 2.5$,
$C_{2f} = 2$	25	19	229.89	260.67	58.12	$C_{2i} = 0.5$,
$C_{2f} = 2.5$	21	14	216.67	253.34	52.45	$P_C = 0.95$,
$C_{2f} = 2.7$	26	17	228.34	256.23	56.98	$P_B = 0.03$
$P_C = 0.5$	29	20	268.56	261.28	66.56	$C_{1i} = 2.5$
$P_C = 0.7$	25	19	246.45	256.21	60.45	$C_{2i} = 0.5$,
$P_C = 0.95$	21	14	216.67	253.34	52.45	$C_{1f} = 0.5$,
$P_C = 1.2$	27	21	255.69	258.73	59.48	$C_{2f} = 2.5$,
$P_C = 1.7$	31	23	273.84	262.56	69.29	$P_B = 0.03$

its target within 26 steps using IPSO-EOPs has been presented in Fig. 12. Figs. 13 and 14 presents the optimal path for IPSO and DE and it needs 29 and 36 steps for all robots. The next position of each robot is evaluated based on its fitness value.

The performance comparison between the three different algorithms IPSO-EOPs, IPSO, and DE:

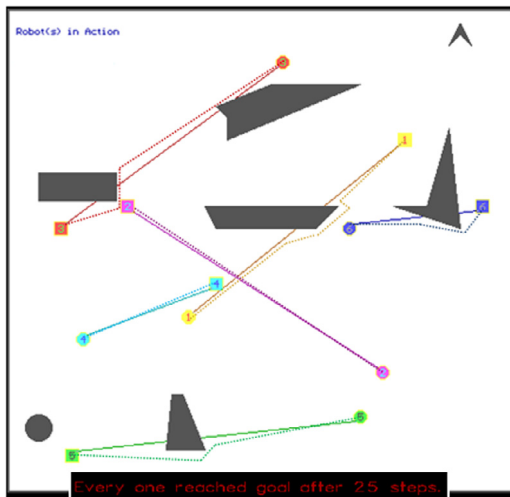
The fitness value for the next position of each robot is presented in Fig. 16. This indicates that robots are generating the

collision free path to their respective target position based on their fitness value. Fig. 16 shows that the fitness value of each robot does not coincide with each other and these indicate that there is no conflict in generating the next position by robots. The initial environment for exercising the mobile robots path planning in the static and dynamic environment has been presented in Fig. 4 and Fig. 9 respectively. Three different algorithms used the same fitness function as presented in Eq. (35) with the same

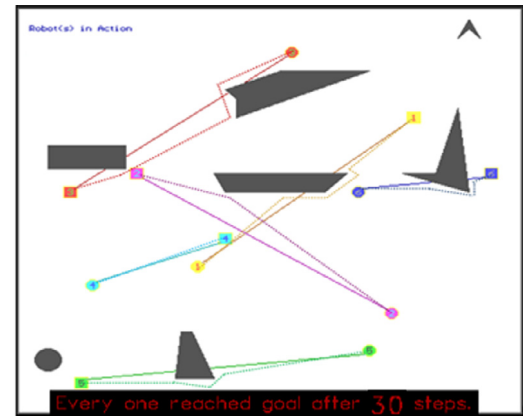
Table 5

Tune of parameters for evaluating the performance of mobile robots path planning through DE.

Parameters	Required steps	Required turn	ATTPD (in pixels)	AUTTD	Execution time (in s)	Other parameters
F = 0.5	37	29	315.95	343.26	85.16	CR=0.65
F = 0.55	34	28	308.73	333.19	82.34	
F = 0.60	31	26	296.72	325.27	81.29	
F = 0.65	30	25	284.28	318.68	79.34	
F = .70	29	24	279.53	313.45	77.74	
F = .75	28	23	255.97	309.37	74.91	
F = .80	30	25	259.35	311.51	76.26	CR=0.70
F = 0.50	36	30	314.28	339.17	81.93	
F = 0.55	33	28	311.25	332.69	80.76	
F = 0.60	32	27	306.82	324.31	79.55	
F = .65	30	26	297.25	317.13	76.21	
F = .70	29	24	279.13	314.89	74.42	
F = .75	30	22	265.18	310.30	71.20	CR=0.75
F = .80	32	24	268.27	313.35	73.75	
F = 0.5	34	30	312.35	332.84	80.43	
F = 0.55	32	27	303.49	321.93	79.23	
F = 0.6	31	27	289.33	319.52	77.34	
F = .65	30	25	276.62	311.29	73.59	
F = .70	29	24	270.15	308.19	72.63	CR=0.80
F = .75	30	23	269.90	299.49	71.78	
F = .80	32	26	271.73	305.10	72.98	
F = 0.5	35	28	305.36	329.16	78.19	
F = 0.55	32	27	297.72	321.76	73.15	
F = 0.6	29	22	278.22	307.58	71.26	
F = .65	30	22	256.82	289.23	70.85	
F = .7	29	21	245.28	281.31	69.19	
F = .75	27	18	236.42	273.89	67.51	
F = .80	29	20	247.34	284.26	72.98	

**Fig. 6.** Six robots arrived at their corresponding goal position after 25 steps through IPSO in static environment.

parameter in 30 iterations to evaluate the next position for each robot. The best fitness value of three algorithms has been offered in Fig. 17. The fitness value of each robot illustrated in Fig. 16 indicates that there is no deadlock situation arises for evaluating the next position of robots. It is observed that the best fitness value is 3.638 in IPSO-EOPs through 21 iterations, but the value obtained by IPSO and DE is 5.195 and 5.815 after 25 and 30 correspondingly. Therefore, IPSO-EOPs is outperformed DE and IPSO in terms of improving convergence rate and avoiding the solution to be trapped at local optima. The optimal number of steps essential for different robots (1 to 6) through the simulation for different algorithms in the static and dynamic environment are offered in Table 6. Table 6 expressed that optimal steps

**Fig. 7.** Six robots arrived at their corresponding goal position after 30 steps through DE in static environment.

essential for IPSO-EOPs is less than IPSO and DE in static as well as the dynamic environment. The total number of optimal steps essential for IPSO-EOPs, IPSO and DE are 21, 25 and 30 for the static environment and it takes 26, 29 and 36 for dynamic environment respectively.

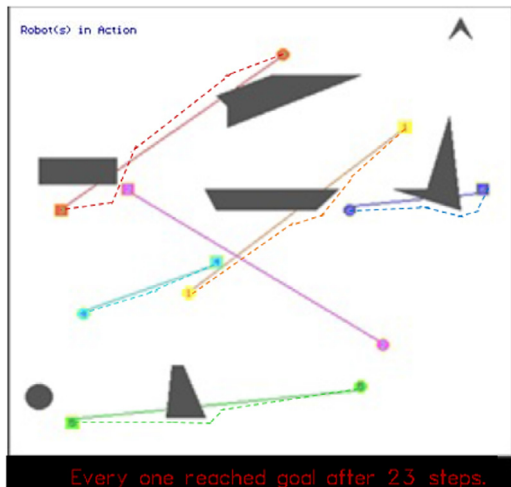
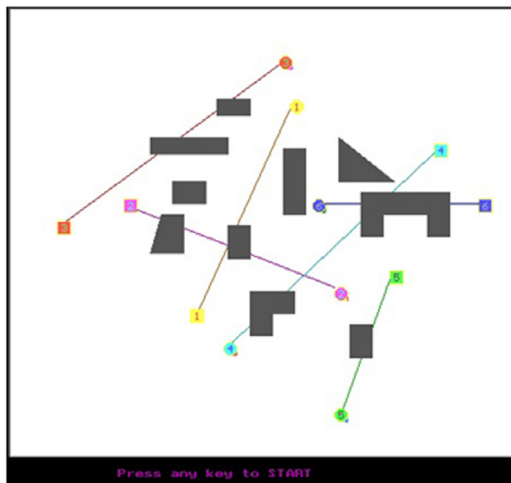
This illustrates that the deviation of the path in IPSO-EOPs is fewer in comparison to supplementary two metaheuristics algorithm DE and IPSO for any number of robots. Here, to avoid the complexity in simulation, we have considered for SIX number of robots and SEVEN obstacles for the static environment and SIX number of robots and TEN obstacles for the dynamic environment. IPSO-EOPs worked for any number of robots and shows that deviation of the path is least in comparison to other algorithms.

AUTTD and stages are illustrated in Fig. 18. It is noticed that AUTTD causes slower converge by reducing the velocity and

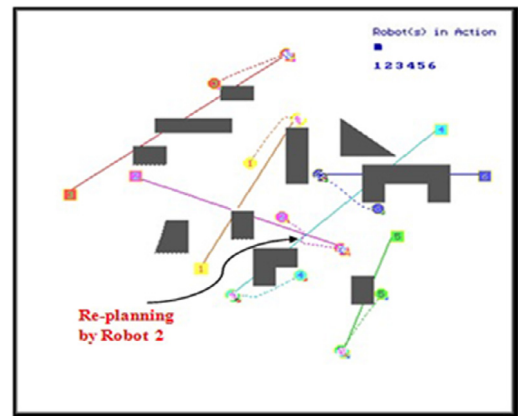
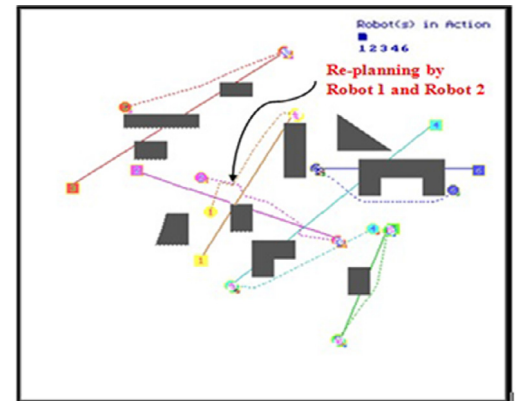
Table 6

Steps requisite for robots to arrive at target present in static and dynamic Environment.

Robot number#	Static environment			Dynamic environment		
	Number of steps needed to target in IPSO-EOPs	Number of steps needed to target in IPSO	Number of steps needed to target in DE	Number of steps needed to target in IPSO-EOPs	Number of steps needed to target in IPSO	Number of steps needed to target in DE
1	21	24	23	26	29	36
2	21	25	31	23	27	30
3	21	23	27	26	29	36
4	11	14	13	18	21	24
5	21	25	25	25	26	28
6	11	13	15	19	24	26

**Fig. 8.** Six robots arrived at their corresponding goal position after 23 steps through ABPSO in static environment.**Fig. 9.** Initial Configuration of dynamic Environment.

slowly dismissed with iteration. If the velocity of the robots increases, it causes quicker the fall-off in AUTTD. Fig. 18 illustrates that if the number of robots is increasing, which may cause showering the convergence speed and it causes the delay in a fall-off in AUTTD. The performance of the result has been investigated through ATPD. The graph has been plotted ATPD with varying the number of robots from 1 to 6 as presented in Fig. 19. This graph presents the deviation of the path through three different meta-heuristic algorithms such as IPSO-EOPs, ABPSO, DE and IPSO and provides smaller ATPD in IPSO-EOPs as compared to IPSO and DE for any quantity of robots. Again, energy utilization during

**Fig. 10.** Six robots are planning their path to avoid dynamic obstacles and robot number 2 is replanning.**Fig. 11.** Six robots are planning their path to avoid dynamic obstacles and robot number 1 and robot number 2 are replanning.

travel of each robot is considered as one of the parameter for evaluating the result of the simulation. The energy utilization is expressed in the form of the number of turns required for each robot. Hence, minimizing the number of turns reduces the energy utilization. Three different algorithms take the different number of turns to arrive at their corresponding goal position as presented in Fig. 20. Fig. 20 illustrate that IPSO-EOPs takes the minimum number of turns as compared to ABPSO, IPSO, and DE. Therefore, IPSO-EOPs takes minimum energy to travel for each robot as compared to IPSO and DE. The simulation is exercised using six numbers of robots for calculating the energy utilization, but the energy utilization is less in the form of the number of turn for any quantity of robots in the planning scheme of the algorithm. Finally, the performance is carried out in the form of the running time necessary to arrive at all robots in their corresponding target position through three algorithms. Fig. 21

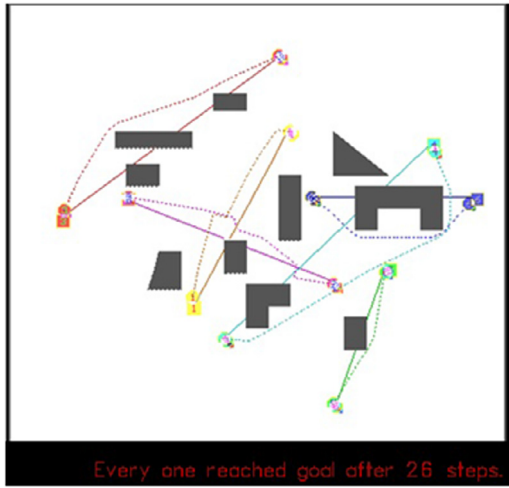


Fig. 12. Six robots arrived at their corresponding goal in 26 steps through IPSO-EOPs.



Fig. 13. Six robots arrived at their corresponding goal in 29 steps through IPSO.

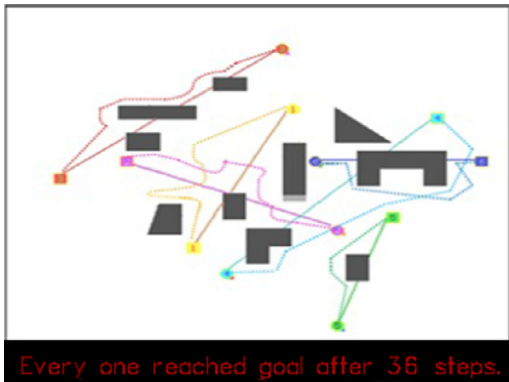


Fig. 14. Six robots arrived at their corresponding goal in 36 steps through DE.

indicates that the time essential for robots to arrive at their corresponding target through three different algorithms and it illustrate that IPSO-EOPs offers less time for all robots to arrive at their target. The uncovered target distance of each robot in every step is presented in Fig. 22. Fig. 22 shows that at step 5 the robot 4 and robot 6 has to cover the same distance, after step 5 robot 6 has to cover more distance than robot 4 and before step 5, robot 6 has to cover less distance than robot 4. This indicates that the path deviation for robot 6 is more than robot 4 as the graph for

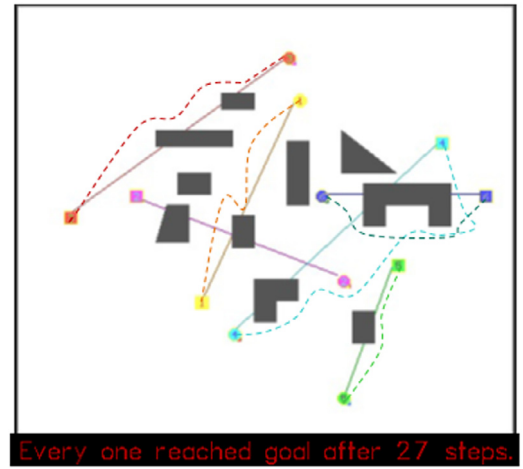


Fig. 15. Six robots arrived at their corresponding goal in 27 steps through ABPSO.

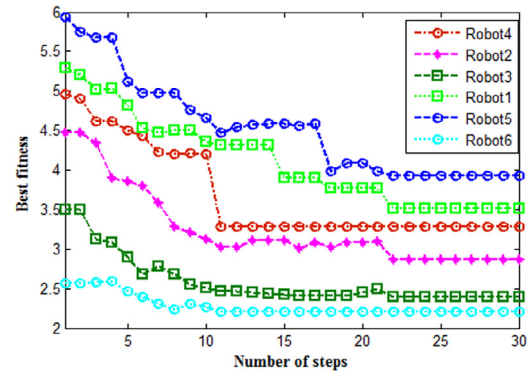


Fig. 16. Fitness value of each robot with the number of steps in IPSO-EOPs.

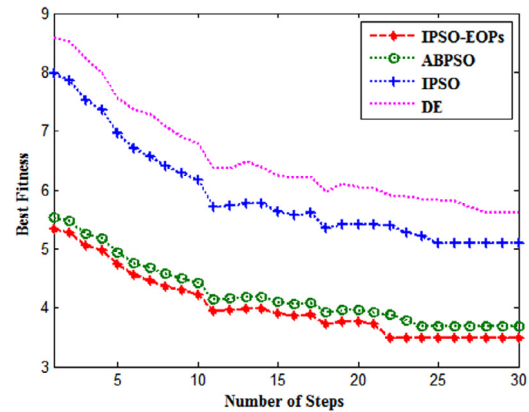


Fig. 17. Best fitness value of IPSO-EOPs, ABPSO, IPSO and DE.

robot 4 is linear. Similarly, for robot 1 and robot 3, initially, the uncovered target distance is almost same, but at step 5 the path deviation is less for robot 1 and after that, both robots will be covered the same distance.

Again the performance is evaluated by plotting ATTPT with n number of robots by fluctuating no of robots (1 to 6) through IPSO-EOPs, IPSO, and DE, and plotted graph has been presented in Fig. 23. The conclusion is drawn from Fig. 23 that IPSO-EOPs possesses less ATTPT in comparison to its competitors for the

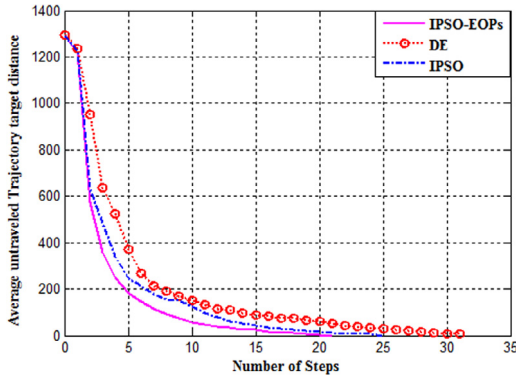


Fig. 18. No. of steps required in different algorithm vs Average untraveled Trajectory target distance.

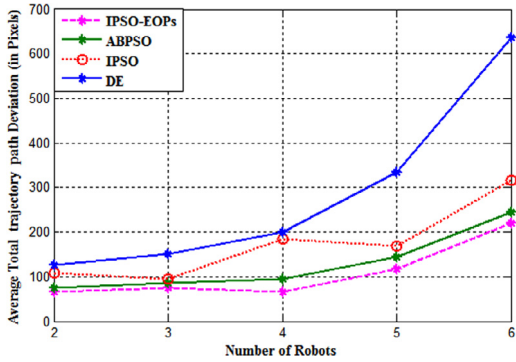


Fig. 19. Average total trajectory path deviation with number of robots for different obstacles and velocity.

different number of robots. This specifies that IPSO-EOPs outperforms than its competitor algorithms in terms of ATTPD by all robots.

The value of weight to the separate fitness function has been exercised through simulation and the value obtained from the simulation has been presented in Table 7. The result shows that, for $\lambda_1 = 1, \lambda_2 = 1, \lambda_3 = 1, \lambda_4 = 0.5, \lambda_5 = 0.5$, it is able to produce an optimal collision free trajectory path with minimum ATTPD, ATTPD and also improve the convergence speed by IPSO-EOPs.

The simulation has been conducted in a different environment for the static and dynamic situations of obstacles. The comparison has been carried out with a static environment and dynamic environment using different algorithms. The simulation result comparison and their corresponding performances have been presented in Table 8. In both, the situation IPSO-EOPs outperforms the remaining its competitor algorithms with respect to all three metrics for different robots.

Again the global performance of algorithms has been evaluated through Eq. (40) for the dynamic and static environment and elaborated in Tables 9 and 10 respectively. It is noted from Table 10 that the overall performance of IPSO-EOPs is less than other competitors which signpost that the cost metrics value used in the performance calculation is less. Hence, IPSO-EOPs has outperformed its competitors.

$$\text{Overall performance} = \mu_1 \times \text{running time} + \mu_2 \times \text{path traveled} + \mu_3 \times \text{No. of turn} + \mu_4 \times \text{No. of steps} \quad (40)$$

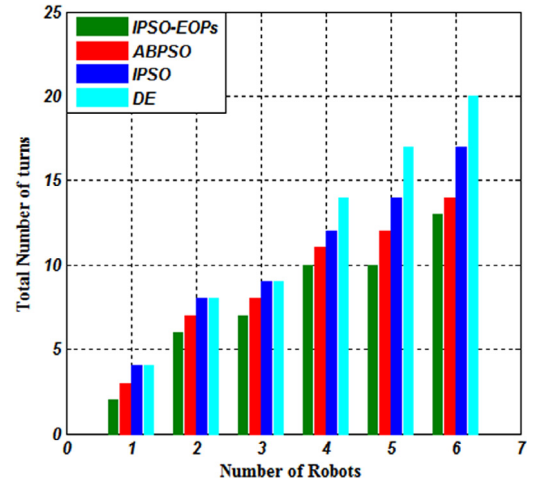


Fig. 20. Number of Turn vs Number of robots in three different algorithms.

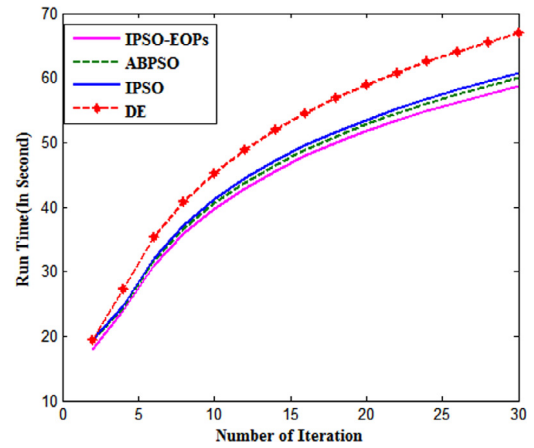


Fig. 21. Run time with Number of iteration for IPSO-EOPs,IPSO and DE.

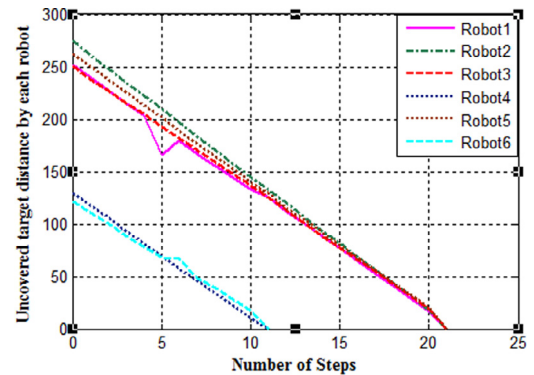


Fig. 22. Uncovered target distance with number of steps.

where μ_1, μ_2, μ_3 and μ_4 are the weight factors of run time, path traveled, number of turns and number of steps respectively and value of the weight factors are as follows: $\mu_1 = 2, \mu_2 = 3, \mu_3 = 4$ and $\mu_4 = 5$.

A comparison has been undertaken in Table 11 in terms of computational time necessary for re-planning by IPSO-EOPs, IPSO, and DE. It is prominent from Table 10 that computational time essential for re-planning by IPSO-EOPs is establishing fewer compared to its competitor algorithms. Simulation has been carried out for large and complex environment with fifteen numbers of

Table 7

Adjustment of weight of the fitness function in Eq 35 through simulation.

Weights					Trajectory Path and convergence rate			
λ_1	λ_2	λ_3	λ_4	λ_5	Average path traveled (in pixels)	Average path deviation (in pixels)	Collision free path(yes/No)	Success rate
*	1	1	0.9	0.7	1678	723	Yes	30(7)
				0.6	1620	684	Yes	30(8)
				0.5	1680	725	Yes	30(10)
				0.4	1240	539	Yes	30(15)
				0.3	1140	460	Yes	30(16)
				0.25	865	420	Yes	30(20)
			0.8	0.7	1695	689	Yes	30(12)
				0.6	1670	625	Yes	30(12)
				0.5	1320	612	Yes	30(18)
				0.4	1210	580	Yes	30(20)
				0.3	1035	435	Yes	30(21)
				0.25	960	396	Yes	30(23)
			0.7	0.7	1578	675	Yes	30(10)
				0.6	1225	620	Yes	30(16)
				0.5	1230	578	Yes	30(20)
				0.4	1045	468	Yes	30(22)
				0.3	945	375	Yes	30(24)
				0.25	722	325	Yes	30(25)
			0.6	0.7	1475	765	Yes	30(9)
				0.6	1135	455	Yes	30(17)
				0.5	1048	425	Yes	30(22)
				0.4	935	368	Yes	30(24)
				0.3	845	320	Yes	30(25)
				0.25	650	226	Yes	30(26)
			0.5	1.0	1045	675	Yes	30(12)
				0.9	860	415	Yes	30(21)
				0.8	956	389	Yes	30(22)
				0.7	720	320	Yes	30(26)
				0.6	645	220	Yes	30(27)
				0.5	612	216	Yes	30(30)
				0.25	678	230	yes	30(24)

Table 8

Comparison of simulation result in static and dynamic environment.

Fig. No	No. robots	No. obstacles	Environment	ATTPT (in pixels)	Time taken (in s)	No. steps required	No. of turn/rotation required	ATTPD (in pixels)
Fig. 5	6	7	Static	612	52.56	21	13	216
Fig. 6	6	7	Static	795	62.37	25	17	318
Fig. 7	6	7	Static	1397	68.25	30	20	637
Fig. 8	6	7	Static	723	58.27	23	14	284
Fig. 12	6	10	Dynamic	734	55.60	26	17	252
Fig. 13	6	10	Dynamic	945	69.63	29	19	456
Fig. 14	6	10	Dynamic	1560	72.57	36	25	872
Fig. 15	6	10	Dynamic	837	62.34	27	17	317

Table 9

Global performance of every robot by IPSO-EOPs in dynamic environment.

Robot #	Required turns	Path traveled	Run time (in s)	Required steps	Global performance
1	3	444.74	50.43	11	1552.51
2	7	734	55.60	15	2416.2
3	9	663.15	53.28	21	2237.01
4	12	289.5	47.34	23	1126.18
5	15	248.84	42.19	24	1010.9
6	17	323.58	49.12	26	1266.98

Table 10

Global Performance of algorithms in terms of cost matrix for Static Environment.

Algorithm	Required turns	Path traveled	Run time (in s)	Required steps	Global performance
IPSO	17	822	62.17	25	2783.34
DE	20	1403.71	68.22	31	4580.74
IPSO-EOPs	13	612	52.56	21	2098.12
ABPSO	14	627	60.27	23	2172.54

robot and twenty numbers of obstacles. The initial world map for conducting the experiment in static and dynamic environment

has been presented in Fig. 24. The optimal trajectory path of proposed algorithm IPSO-EOPs by considering a complex static map has been presented in Fig. 25. Fig. 25 shows that all robot reached

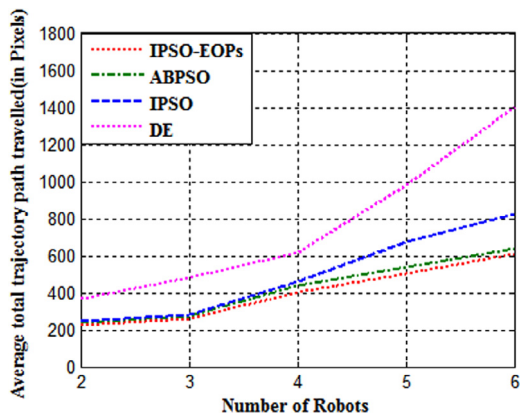


Fig. 23. Average trajectory path traveled with Number of robots.

Table 11
Computational time required for re-planning by IPSO-EOPs, IPSO, and DE.

Time in s	Re-planning			
	IPSO-EOPs	IPSO	DE	ABPSO
5 obstacles	0.43	0.65.	0.72	0.53
7 obstacles	0.47	0.67	0.74	0.58
10 obstacles	0.49	0.75	0.78	0.61

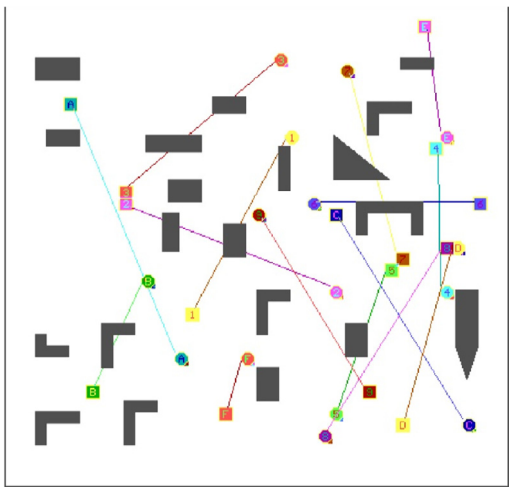


Fig. 24. Initial environment for 15 robots and 20 obstacles.

in their destination in 45 steps. The optimal for other algorithm such as IPSO, ABPSO and DE has been presented in Fig. 26, Fig. 27 and Fig. 28 respectively. The number of steps necessary to reach all robot in their destination through IPSO, ABPSO and DE are 57, 64 and 68 respectively. The result reveals that the superiority proposed algorithm IPSO-EOPs over its competitors. Similarly, the experiment also conducted in dynamic environment and result of the trajectory path has been presented in Fig. 29. Fig. 29 shows the smoothness of the path and all robot reached in their target position in 56 steps through IPSO-EOPs. The trajectory path of its competitor's algorithm has been presented in Fig. 30, Fig. 31 and Fig. 32. The performance has been analyzed for static and dynamic scenario and it has been demonstrated in Table 11. The result as reflected in Table 12 reveals that the superiority of the proposed algorithm IPSO-EOPS over its competitors.

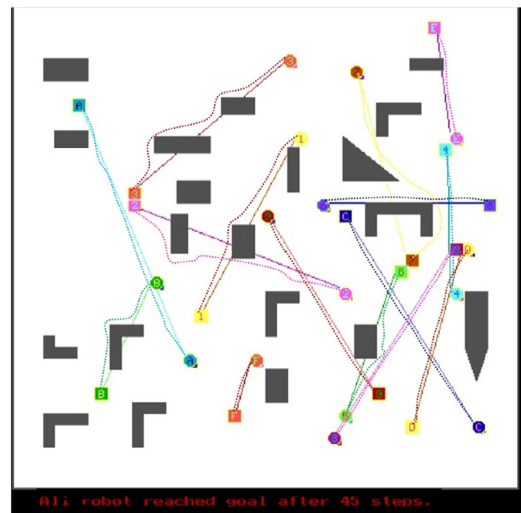


Fig. 25. Fifteen robots arrived at their corresponding goal position after 45 steps through IPSO-EOPs in the static environment.

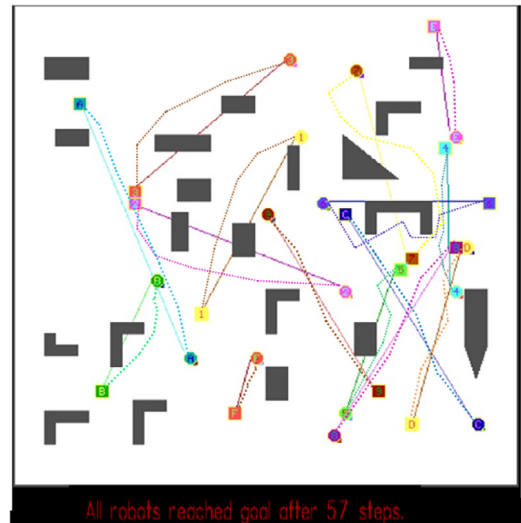


Fig. 26. Fifteen robots arrived at their corresponding goal position after 57 steps through IPSO in the static environment.

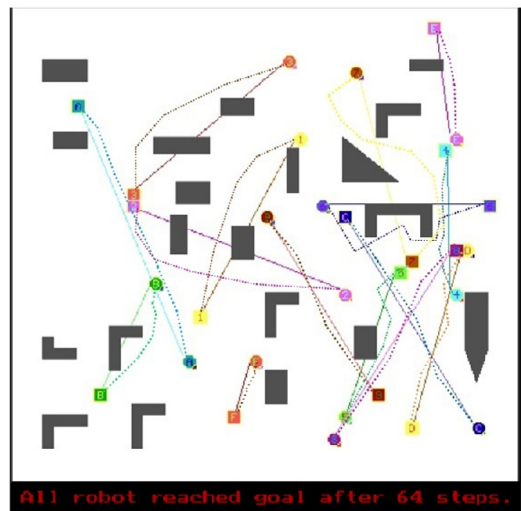
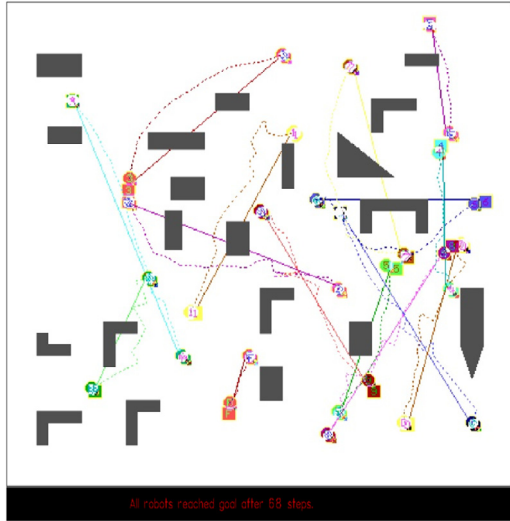
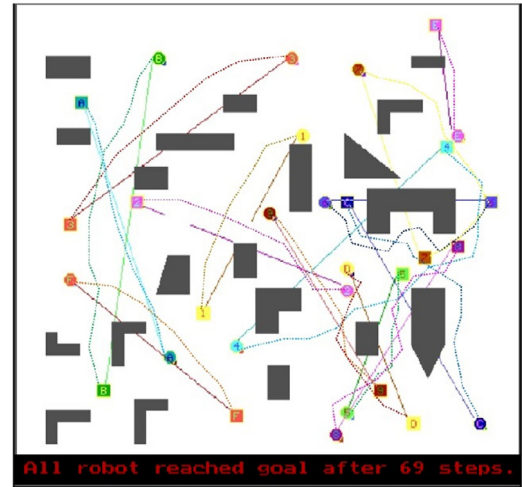
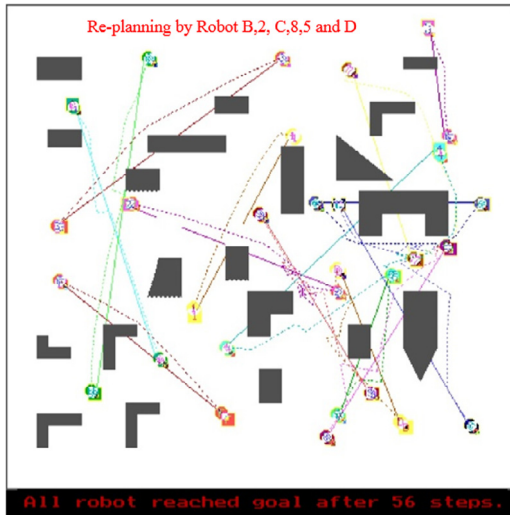


Fig. 27. Fifteen robots arrived at their corresponding goal position after 64 steps through ABPSO in the static environment.

Table 12

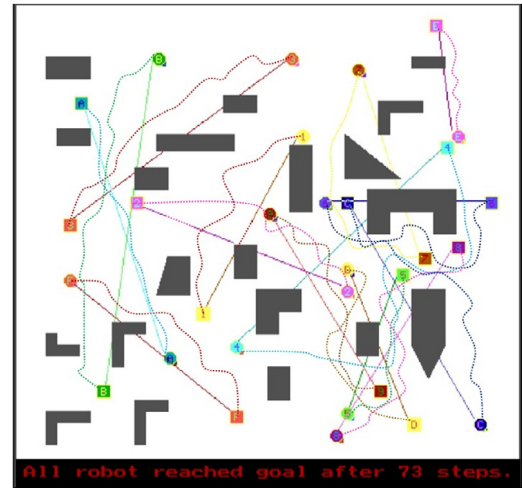
Comparison of simulation result in static and dynamic environment for 15 numbers of robot and 20 numbers of obstacles.

Fig. No	No. robots	No. obstacles	Environment	ATTPT (in Pixels)	Time taken (in s)	No. steps required	No. of turn/rotation required	ATTPD (in Pixels)
	15	20	Static	1542	133.65	45	35	544
Fig. 25	15	20	Static	2017	157.92	57	42	805
Fig. 26	15	20	Static	2672	169.72	64	51	1147
Fig. 27	15	20	Static	3508	173.62	68	56	1619
Fig. 29	15	20	Dynamic	1867	141.53	56	46	648
Fig. 30	15	20	Dynamic	2389	177.04	69	59	1146
Fig. 31	15	20	Dynamic	3243	179.14	73	61	1752
Fig. 32	15	20	Dynamic	3957	183.42	76	65	2207

**Fig. 28.** Fifteen robots arrived at their corresponding goal position after 68 steps through DE in the static environment.**Fig. 30.** Fifteen robots arrived at their corresponding goal in 69 steps through IPSO.**Fig. 29.** Fifteen robots arrived at their corresponding goal in 56 steps through IPSO-EOPs.

8. Experiment on Khepera II robots

The description of Khepera II and its numerical values of the parameter has been presented in Table 13. The sensors are equipped in the kephera-II mobile robot in an anticlockwise manner and it has been number as 0 to 7 with the leftmost

**Fig. 31.** Fifteen robots arrived at their corresponding goal in 73 steps through ABPSO.

sensors designated 0 and the rightmost by 7. These parameters are exercised for conducting the experiment and robot is controlled through the personal computer via com port. The experiment is carried out through two Khepera-II mobile robots. The motor of wheels gets the control to operate the movement of the robot in a particular direction and this control signal is generated through the sensor value obtained in the world map. A control program that regulates the next position of a robot from an existing position using the proposed optimization technique.

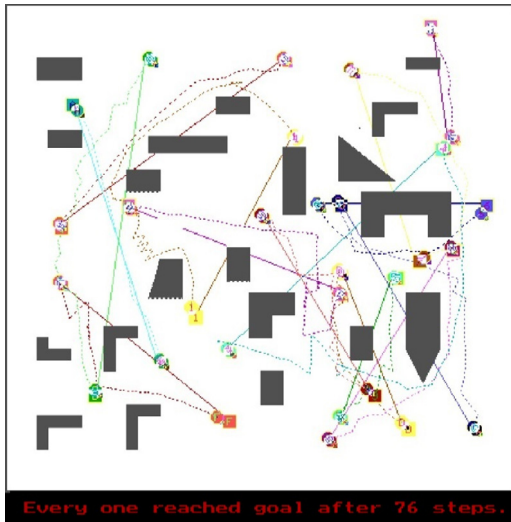


Fig. 32. Fifteen robots arrived at their corresponding goal in 76 steps through DE.

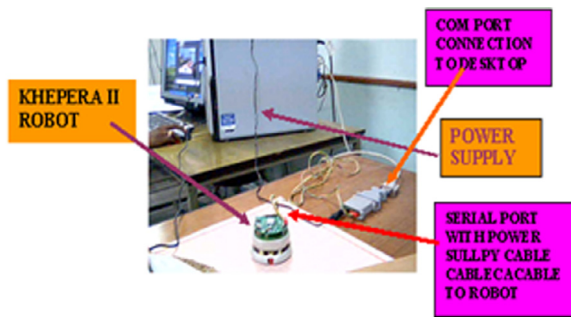


Fig. 33. Khepera network and its accessories.

The algorithm runs on the attached desktop machine and its set up with accessories has been illustrated in Fig. 33 for carrying out the experiment.

The experiment has been conducted on the real platform through the khepera II robot and the environment considered for the experiment to be conducted is illustrated in Fig. 34. The environment consisted of Seven different shaped static obstacles and two Khepera-II robots. The source and target for each robot have been signified with unlike color code in the environment. Three meta-heuristic algorithms are exercised through the Khepera robots and the result of the optimal collision free trajectory path has been offered in Fig. 35. The inference pinched from Fig. 35 that the projected algorithm IPSO-EOPs provides superior optimization as compared to other competitors in the form of the path traveled, the time necessary to arrive at their corresponding position and utilization of energy in the form of the number of turns. The result of the performance generated through three meta-heuristic algorithms has been illustrated in Table 13. Table 13 indicates that IPSO-EOPs is outperforming DE and IPSO. Similarly, the experiment has been exercised in a dynamic environment with IPSO-EOPs, IPSO, and DE. The optimal trajectory path has illustrated using the different color markers for different algorithms and it has been illustrated in Fig. 36. The performance result has been presented in Table 14. It indicates that IPSO-EOPs outperform IPSO and DE in the form of Path travel, Energy utilization and time need every robot to reach their corresponding goal.

Finally, comparison has been made with real platform and simulation result for two robots and the same thing has been

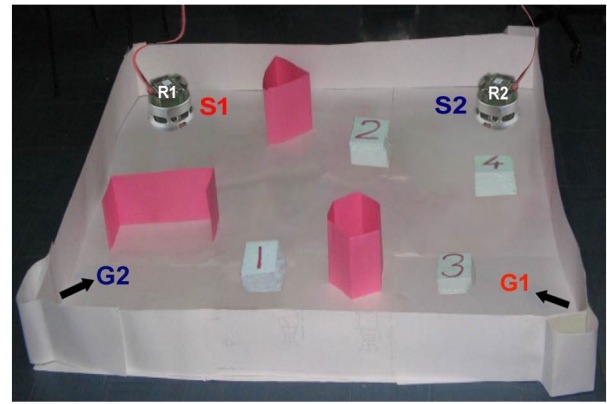


Fig. 34. Initial environment for conducting experiment in static environment..

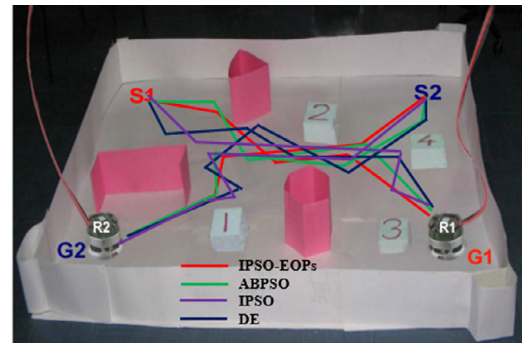


Fig. 35. Optimal trajectory path representation of different algorithm for multi-robot path planning in static environment using Khepera-II is represented by different color code.

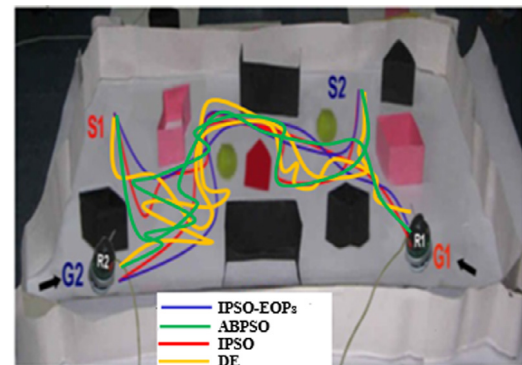


Fig. 36. Optimal trajectory path representation of different algorithm for multi-robot path planning in dynamic environment using Khepera-II is represented by different color code.

explained in Table 15. it is prominent from Table 15 that IPSO-EOPs provided less error than its competitors in form of total trajectory path traveled from their predefined initial position to target position, less error indicates that IPSO-EOPs worked the same manner in simulation and experiment with real robots.

9. Comparison of the proposed approach with state of arts

9.1. Comparison study with Ref. [8]

This section provides a comparative study of the proposed approach of IPSO-EOPs with the existing approach suggested by Ali et al. [8] for navigation of multi-robot through the shortest

Table 13

Description of Khepera-II Mobile robot.

Sl. No	Description	Parameter Type	Numerical Values
1	Diameter	Khepera-II	8 cm
2	Sensors	infrared proximity	8
3	Motor	caster wheel	2
4	Sensor range	>5 cm	0
		<=2cm	1023
5	memory	Flash	256 KB
6	Microprocessor	Motorola 68331	25 MHZ
8	Height	Khepera-II	130 mm
9	Sensor	Height from ground	30 mm
10	Weight	Khepera-II	80 g
11	Speed	Maximum	0.5 m/s
		Minimum	0.02 m/s

Table 14

Analysis of performance in dynamic and static Environment.

Algorithm	Static Environment		Dynamic environment	
	Turn necessary	Run time	Turn necessary	Run time
IPSO-EOPs	06	57.32	10	59.15
IPSO	08	69.17	11	72.78
DE	10	74.39	20	81.34
ABPSO	07	62.59	09	64.72

Table 15

Comparison between experimental and simulation result for two robots using IPSO-EOPs.

Different techniques	Optimal Path (in pixel) in Static environment			Optimal path (in pixel) in dynamic environment		
	In simulation	In experimental model	% error in simulation and experimental	In simulation	In experimental model	% error in simulation and experimental
IPSO-EOPs	273	315	13.33%	288	348	17.24
IPSO	357	420	15%	382	468	18.37
DE	389	484	19.62%	406	525	22.66
ABPSO	285	330	13.63	307	372	17.47

distance algorithm. The proposed algorithm evaluates the collision free shortest trajectory path using the existing position and orientation of other algorithms. The simulation result of the proposed algorithm has been compared with the existing algorithm to verify the robustness and effectiveness of the proposed algorithm IPSO-EOPs. The outcomes of the simulation result reveal that the existing technique only optimized the path length but did not evaluate the energy consumption in form of turns, the time required for every robot to reach their respective location and not evaluated path deviation. The comparison between the proposed algorithm IPSO-EOPs and the existing algorithm has been presented in Table 16.

9.2. Comparison with Ref. [53]

In this section, the comparison of IPSO-EOPs with Adaptive bare-bones particle swarm optimization algorithm [53] has been carried out to verify the effectiveness of the proposed algorithm. Zhang et al. studied an improved BBPSO algorithm with adaptive disturbance (ABPSO). They used stochastic process theory to analyze the convergence of ABPSO, by regarding each particle's position as a stochastic vector. In the multi-robot path planning, the velocity of the robot should be carefully calculated to decide the next collision free optimal position. The author [53] has eliminated the velocity update equation which drastically deteriorates the performance of ABPSO in the field of path planning. The comparison has been carried out with ABPSO in the form of simulation and experiments in the real platform. The simulation has been conducted on the same environment as IPSO-EOPs and the result of ABPSO for the static and dynamic environment has been presented in Figs. 8 and 15 respectively. Again the performance of the ABPSO has been compared with the results of

IPSO-EOPs, IPSO and DE in the terms of best fitness value, ATPD, Total number turns, runtime and ATPT in Figs. 17, 19, 20, 21 and 23 respectively. It is perceived that IPSO-EOPs outperforms its competitors. Similarly, the performance of ABPSO has been compared with IPSO-EOPs, IPSO and DE. Again the comparison has been exercised in the real platform and experimental has been presented in Figs. 35 and 36 for static and dynamic environment respectively. The result presented in Figs. 35 and 36 shows that IPSO-EOPs outperforms its competitors in both environments. The performance of the experimental result has been compared and presented in Tables 14 and 15. It illustrates that IPSO-EOPs is performed better than its competitors.

9.3. Comparison A* algorithm [54] with IPSO-EOPs

The efficacy of A* algorithm has been verified with the proposed algorithm IPSO-EOPs by considering a highly complex environment. The environment under consideration consists of 15 numbers of robots and 20 number of obstacles. The path length has been considered as an only objective function for pair comparison from the existing algorithm. The scenario for exercising the algorithm has been presented in Figs. 37 and 38. It is evident from the final scenario that A* algorithm takes more number of steps and smoothness of the path not visible as compared to IPSO-EOPs. The performance of the A* and IPSO-EOPs algorithm has been evaluated and presented in Table 17. The result as reflected in Table reveals that the superiority of the proposed algorithm over A*.

10. Conclusion and future direction

A hybridization of IPSO-EOPs algorithm has been proposed to enhance the exploration and exploitation capability for better convergence. A case study on multiple mobile robot path

Table 16
Comparison proposed algorithm with work of Ali et al. [8].

Number of robots	Benchmarks	Reference [8]		Proposed algorithm
		ROA (reciprocal orientation algorithm)	SDA (shortest distance algorithm)	IPSO-EOPs
2	length of Path(in Pixels)	673	663	557
	Required turns	Not calculated	Not calculated	03
	Deviation of Path(in Pixels)	Not calculated	Not calculated	82
	Required steps	Not calculated	Not calculated	12
	Time taken(in s)	Not calculated	Not calculated	17.25
4	length of Path(in Pixels)	708	689	598
	Required turns	Not calculated	Not calculated	10
	Deviation of Path(in Pixels)	Not calculated	Not calculated	100
	Required steps	Not calculated	Not calculated	18
	Time taken(in s)	Not calculated	Not calculated	19.39
6	length of Path(in Pixels)	736	705	612
	Required turns	Not calculated	Not calculated	13
	Deviation of Path(in Pixels)	Not calculated	Not calculated	210
	Required steps	Not calculated	Not calculated	21
	Time taken(in s)	Not calculated	Not calculated	52.56

Table 17
Comparison of simulation result considering path length only as an objective function.

Algorithms	No. robots	No. obstacles	Environment	ATTPT (in Pixels)	Time taken (in s)	No. steps required	No. of turn/rotation required	ATTPD (in Pixels)
IPSO-EOPs	15	20	Static	1957	143.74	51	39	747
A*	15	20	Static	3798	188.27	77	67	1685

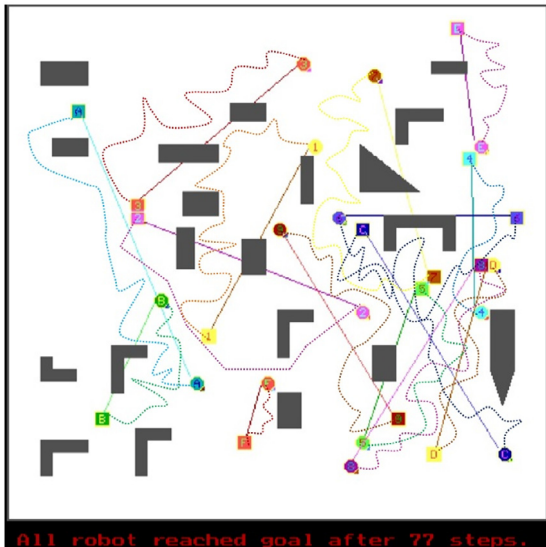
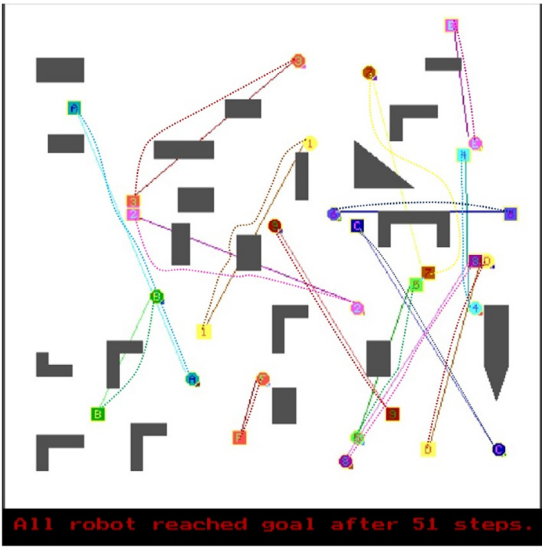


Fig. 37. Fifteen robots arrived at their corresponding goal in 51 steps through IPSO-EOPs.

Fig. 38. Fifteen robots arrived at their corresponding goal in 77 steps through A* algorithm.

planning has been undertaken using the proposed technique IPSO-IGSA to compute the optimal trajectory path from the pre-define initial position to target position for each robot in the environment by avoiding the static and dynamic obstacles. The experiments, undertaken on simulation, through the Khepera-II robot reveal that proposed algorithm IPSO-EOPs outperforms over DE and IPSO with respect to path deviation, path traveled, and optimized energy in the form of the number of turns needed for each robot to arrive at their respective target. The experiments performed with the Khepera-II robot indicates that IPSO-EOPs outperforms other realization in the real platform, thereby justifying the efficiency of the proposed algorithm. Finally, the comparison has been exercised between the proposed

method IPSO-EOPs and previously proposed algorithm [8]. The comparison result shows that IPSO-EOPs is performing better result as compared to the existing algorithm [8] for the irrespective number of robots. Again, the comparison has been carried out against several test functions between the proposed algorithm and the existing algorithm [53]. The outcome of the proposed algorithm is satisfactory. In the future, visual perception based multi-robot path planning will be carried out in the dynamic environment.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRedit authorship contribution statement

P.K. Das: Conceptualization, Methodology, Software, Writing - review & editing, **P.K. Jena:** Data curation, Writing - original draft, Visualization, Investigation, Validation.

References

- [1] M.A. Shoushtary, H.H. Nasab, M.B. Fakhrazad, Team robot motion planning in dynamics environments using a new hybrid algorithm (honey bee mating optimization-tabu list), *Chinese J. Eng.* 2014 (2014) 1–8.
- [2] A. Konar, *Artificial Intelligence and Soft Computing: Behavioral and Cognitive Modeling of the Human Brain*, first ed., CRC Press, 1999.
- [3] I. Duleba, J.Z. Sasiadek, Nonholonomic motion planning based on Newton algorithm with energy optimization, *IEEE Trans. Control Syst. Technol.* 11 (3) (2003) 355–363.
- [4] J. Xiao, Z. Michalewicz, L. Zhang, K. Trojanowski, Adaptive evolutionary planner/navigator for mobile robots, *IEEE Trans. Evol. Comput.* 1 (1) (1997) 18–28.
- [5] K. Jose, D.K. Pratihari, Task allocation and collision-free path planning of centralized multi-robots system for industrial plant inspection using heuristic methods, *Robot. Auton. Syst.* 80 (2016) 34–42.
- [6] K.G. Shin, Q. Zheng, Minimum time collision free trajectory planning for dual robot systems, *IEEE Trans. Robot. Autom.* 8 (5) (1992) 641–644.
- [7] M. Moll, L.E. Kavraki, Path-planning for minimal energy curves of constant length, in: *Proc. IEEE International Conference on Robotics & Automation*, 2004, pp. 2826–2831.
- [8] A.A. Ali, A.T. Rashid, M. Frasca, L. Fortuna, An algorithm for multi-robot collision-free navigation based on shortest distance, *Robot. Auton. Syst.* 75 (2016) 119–128.
- [9] B.K. Olewi, H. Roth, B.I. Kazem, A hybrid approach based on ACO and GA for multi objective mobile robot path planning, *App. Mech. Mater.* 527 (2014) 203–212.
- [10] Y. Guo, L.E. Parker, A distributed and optimal motion planning approach for multiple mobile robots, in: *Proc. IEEE International Conference on Robotics and Automation (ICRA'02)*, 2002, pp. 2612–2619.
- [11] S.J. Buckley, Fast motion planning for multiple moving robots, in: *Proc. IEEE Int. Conf. on Robotics and Autom.*, 1989, pp. 1419–1424.
- [12] X. Wei, Robot path planning based on simulated annealing & artificial neural networks, *Res. J. Appl. Sci. Eng. Technol.* 6 (1) (2013) 149–155.
- [13] P. Bhattacharya, M.L. Gavrilova, Roadmap-based path planning using the voronoi diagram for a clearance-based shortest path, *IEEE Robot. Autom. Mag.* 15 (2) (2008) 58–66.
- [14] P.K. Das, A. Konar, R. Laishram, Path planning of mobile robot in unknown environment, *Int. J. Comp. Comm. Tech.* 1 (2010) 26–31.
- [15] R. Regele, P. Levi, Cooperative multi-robot path planning by Heuristic priority adjustment, in: *Proc. IEEE/RSJ Int Conf Intell. Robots Syst.* 2006, pp. 5954–5959.
- [16] X. Liu, D. Gong, A comparative study of A-star algorithms for search and rescue in perfect maze, in: *Proc. International Conference on Electric Information and Control Engineering (ICEICE)*, 2011, pp. 24–27.
- [17] P.K. Das, H.S. Behera, S. Das, H.K. Tripathy, B.K. Panigrahi, S.K. Pradhan, A hybrid improved PSO-DV algorithm for multi-robot path planning in a clutter environment, *Neurocomputing* 207 (2016) 735–753.
- [18] M. Duguleana, G. Mogan, Neural networks based reinforcement learning for mobile robots obstacle avoidance, *Expert Syst. Appl.* 62 (2016) 104–115.
- [19] M. Ju, S. Wang, J. Guo, Path planning using a hybrid evolutionary algorithm based on tree structure encoding, *Sci. World J.* 2014 (2014).
- [20] C. Son, Intelligent rule-based sequence planning algorithm with fuzzy optimization for robot manipulation tasks in partially dynamic environments, *Inform. Sci.* 342 (2016) 209–221.
- [21] S.E. Ferik, M.T. Nasir, U. Baroudi, A behavioral adaptive fuzzy controller of multi robots in a cluster space, *Appl. Soft Comput.* 44 (2016) 117–127.
- [22] C. Purcaru, R.E. Precup, D. Iercan, L.O. Fedorovici, R.C. David, F. Dragan, Optimal robot path planning using gravitational search algorithm, *Artificial Intelligence* 10 (2013) 1–20.
- [23] C. Purcaru, R.E. Precup, D. Iercan, L.O. Fedorovici, R.C. David, Hybrid PSO-GSA robot path planning algorithm in static environments with danger zones, in: *Proc. 17th International Conference on System Theory, Control and Computing (ICSTCC)*, IEEE, 2013, pp. 434–439.
- [24] C. Purcaru, R.E. Precup, D. Iercan, L.O. Fedorovici, E.M. Petriu, E.I. Voisan, Multi-robot GSA and PSO based optimal path planning in static environments, in: *Proc. IEEE International Conference on Robot Motion and Control (RoMoCo)*, 2013, pp. 197–202.
- [25] E. Masehian, D. Sedighzadeh, Multi-objective PSO-and NPSO based algorithms for robot path planning, *Adv. Electr. Comput. Eng.* 10 (4) (2010) 69–76.
- [26] Y. Zhang, D.W. Gong, J.H. Zhang, Robot path planning in uncertain environment using multi-objective particle swarm optimization, *Neurocomputing* 103 (2013) 172–185.
- [27] J. Chakraborty, A. Konar, L.C. Jain, Uday Kumar Chakraborty, Cooperative multi-robot path planning using differential evolution, *J. Intell. Fuzzy Systems* 20 (1,2) (2009) 13–27.
- [28] G.C. Luh, W.C. Cheng, Behavior-based intelligent mobile robot using immunized reinforcement adaptive learning mechanism, *Adv. Eng. Informat.* 16 (2) (2002) 85–98.
- [29] P.K. Das, S.K. Pradhan, S.N. Patro, B.K. Balabantaray, Artificial immune system based path planning of mobile robot, in: *Proc. Soft Computing Techniques in Vision Science*, Springer, SCI 395, 2012, pp. 195–207.
- [30] P. Bhattacharjee, P. Rakshit, I. Goswami, A. Konar, A.K. Nagar, Multi-robot path-planning using artificial bee colony optimization algorithm, in: *Proc. Third World Congress Nature and Biologically Inspired Computing (NaBiC)*, IEEE, 2011, pp. 219–224.
- [31] P.K. Das, H.S. Behera, B.K. Panigrahi, A hybridization of an improved particle swarm optimization and gravitational search algorithm for multi-robot path planning, *Swarm Evol. Comput.* 28 (2016) 14–28.
- [32] A.H. Paniagua, A., M.A. Vega-Rodríguez, Applying the MOVNS (multi-objective variable neighborhood search) algorithm to solve the path planning problem in mobile robotics, *Expert Syst. Appl.* 58 (2016) 20–35.
- [33] S. Das, A. Mukhopadhyay, A. Roy, A. Abraham, B.K. Panigrahi, Exploratory power of the harmony search algorithm analysis and improvements for global numerical optimization, *IEEE Trans. Syst. Man Cybern.* 41 (1) (2011) 89–106.
- [34] X. Zhang, X. Zheng, R. Cheng, J. Qiu, Y. Jin, A competitive mechanism based multi-objective particle swarm optimizer with fast convergence, *Inform. Sci.* 427 (2018) 63–76.
- [35] Q. Tang, S. Gao, Y. Liu, F. Yu, Infrared image segmentation algorithm for defect detection based on FODPSO, *Infrared Phys. Technol.* 102 (2019) 103051.
- [36] F. Di Martino, S. Sessa, PSO Image thresholding on images compressed via fuzzy transforms, *Inform. Sci.* 506 (2020) 308–324.
- [37] Z.J. Wang, Z.H. Zhan, W.J. Yu, Y. Lin, J. Zhang, T.L. Gu, J. Zhang, Dynamic group learning distributed particle swarm optimization for large-scale optimization and its application in cloud workflow scheduling, *IEEE Tans. cybern.* (2019).
- [38] M.J. Mahmoodabadi, Z.S. Mottaghi, A. Bagheri, HEPPO: High exploration particle swarm optimization, *Inform. Sci.* 273 (2014) 101–111.
- [39] R. Burman, S. Chakrabarti, S. Das, Democracy-inspired particle swarm optimizer with the concept of peer groups, *Soft Comput.* 21 (12) (2017) 3267–3286.
- [40] Z. Zhan, J. Zhang, Y. Li, H.S. Chung, Adaptive particle swarm optimization, *IEEE Trans. Syst. Man Cybern. B* 39 (6) (2009) 1362–1381.
- [41] Z.H. Zhan, J. Zhang, Y. Li, Y.H. Shi, Orthogonal learning particle swarm optimization, *IEEE Trans. Evol. Comput.* 15 (6) (2011) 832–847.
- [42] R. Kundu, R. Mukherjee, S. Das, Modified particle swarm optimization with switching update strategy, in: *Proc. International Conference on Swarm, Evolutionary, and Memetic Computing*, Springer, Berlin, Heidelberg, 2012, pp. 644–652.
- [43] Z. Bien, J. Lee, A minimum-time trajectory planning method for two robots, *IEEE Trans. Robot. Autom.* 8 (3) (1992) 443–450.
- [44] A. Ratnaweera, S.K. Halgamuge, Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficient, *IEEE Trans. Evol. Comput.* 83 (2004) 240–255.
- [45] J.J. Liang, A.K. Qin, S. Baskar, Comprehensive learning particle swarm optimizer for global optimization of multimodal functions, *IEEE Trans. Evol. Comput.* 10 (3) (2006) 281–295.
- [46] M.H. Sadafi, R. Hosseini, H. Safikhani, A. Bagheri, M.J. Mahmoodabadi, Multi-objective optimization of solar thermal energy storage using hybrid of particle swarm optimization, multiple crossover and mutation operator, *Int. J. Eng. Trans. B* 243 (2011) 367–376.
- [47] M. Lovbjerg, T.K. Rasmussen, T. Krink, Hybrid particle swarm optimizer with breeding and subpopulations, in: *Proceedings of Genetic, Evolutionary Computation*, 2001, pp. 469–476.
- [48] P.J. Angeline, Using selection to improve particle swarm optimization in: *Proc. IEEE Congress on Evolutionary Computation*, Anchorage, AK, 1998, pp. 84–89.
- [49] W.D. Chang, A multi-crossover genetic approach to multivariable PID controllers tuning, *Expert Syst. Appl.* 33 (2007) 620–626.
- [50] B. Akay, D. Karaboga, A modified artificial bee colony algorithm for real-parameter optimization, *Inf. Sci.* 192 (2012) 120–142.

- [51] D. Karaboga, An Idea Based on Honey Bee Swarm for Numerical Optimization, Technical Report TR06, Erciyes University, Engineering Faculty, Computer Engineering Department, 2005.
- [52] S. Das, A. Konar, U.K. Chakraborty, Improving particle swarm optimization with differentially perturbed velocity, in: Proc. 7th Annual Conference on Genetic and Evolutionary Computation, ACM, 2005, pp. 177–184.
- [53] Y. Zhang, D.W. Gong, X.Y. Sun, N. Geng, Adaptive bare-bones particle swarm optimization algorithm and its convergence analysis, *Soft Comput.* 18 (7) (2014) 1337–1352.
- [54] P.K. Das, S.K. Pradhan, H.K. Tripathy, P.K. Jena, Improved real time A*-fuzzy controller for improving multi-robot navigation and its performance analysis, *Int. J. Data Sci.* 2 (2) (2017) 105–137.